济南大学

本科毕业设计



基于 Web 的 OJ 在线判题系统设计与实现基于 Web 的 OJ 在线判题系统的设计和实现

学　　院	信息科学与工程学院
专　　业	计算机科学与技术
班　　级	**计科 2202 班**
学　　生	张鹏飞
学　　号	202231222024
指导教师	蔺永政

二〇二六年 六月 八日

济南大学毕业论文（设计）诚信承诺书

本人声明所提交的毕业论文（设计）是本人在导师指导下完成的研究成果，没有抄袭、剽窃他人之行为，也未将他人的作品作为自己的研究成果使用。除文中特别加以标注和致谢之处外，本文不包含他人已发表或撰写过的研究成果，也不存在任何他人未予公开的信息。与我一起工作的同志对本研究所做的一切贡献，都在论文中已经做了明确的说明，并且表达了谢意。

学生签名：　　日期：　　年　　月　　日

济南大学毕业论文（设计）使用授权书

本人同意济南大学将本人的毕业论文（设计）的复印件、电子文档等材料供有关单位或个人查阅、借阅。本人电子文档的内容和纸质论文的内容相一致。除保密

期间的保密论文外，允许论文被查阅、借阅，可以公开（刊登）论文的全部或者部分内容。的公布（包括刊登）授权济南大学教务处办理。

学生签名：　　导师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

针对校内已有的程序设计评测系统架构落后、高并发时容易出现宕机、缺少部分功能等状况，本文提出要开发并实现一个高性能、安全、易于扩展的济南大学 ACM 在线评测系统。

本设计采用微服务架构，基于 Spring Cloud、Vue 3 及 Nacos 构建全栈体系。研究方法主要是分布式协作和安全隔离，用 RabbitMQ 异步队列来应对瞬时高压的问题，用 REST/gRPC 接口对接 go-judge 裁判沙箱来达到测试环境的安全隔离目的。本文详细介绍了学校复杂的网络环境对于 WebVPN 和多层代理的适配过程，用 Redis 分布式锁来保证高并发的时候数据的一致性。工程开发过程中用到 GitHub 持续集成、版本控制，共提交了 225 次变更。系统的核心工程规模比较大，后端的核心业务逻辑代码量为 35000 行，前端的交互和逻辑代码量为 22000 行（不包括配置文件和日志）。

从实际情况看，本系统顺利过渡至所有用户的个人资料中，成功举办了“济南大学第八届程序设计新生赛”比赛。系统在比赛过程中运作良好，最大请求数为 350 QPS，一天之内可处理 4270 条提交信息。由此可见，此系统很好地解决以前的老系统经常出现卡顿现象的问题，有特判以及交互判题的功能，在校内网这种特殊的环境下成功做到任务分离及分配，未使用过多的计算机设备。

关键词： **在线评测系统；微服务；沙箱隔离；高并发；异步处理**

ABSTRACT

Due to the problems of the original programming evaluation system at the university, such as an old architecture, instability under high concurrency, and missing various functions, this Design aims to build and run a high-performance, secure and extensible "University of Jinan ACM Online Judge System".

Spring Cloud and Vue 3 will be employed in this Design for the construction of microservices. Distribution Collaboration and Security Isolation of Research Methods: RabbitMQ is used to clip peak tasks, and REST/gRPC is employed to communicate with the go-judge sandbox for environment isolation. Due to the complexity of the campus network, a WebVPN and a multi-layer proxy will be employed in this Design, and distributed locks of Redis will be used to ensure data consistency. 225 good commits have been made to GitHub for continuous integration in the development stage. The range of the system is relatively large, and the main back-end business logic is about 35,000 lines long and the front-end interaction logic is approximately 22,000 lines.

According to the actual results, the system has successfully completed the transfer of all user data and supported the "8th Freshman Programming Contest of University of Jinan". The system worked normally on the day of the event, reached a peak of 350 QPS, and had accumulated more than 4,270 submissions in one day. Therefore, it can be concluded that the system has solved the lag problem in the old version and can support special judgement and interactive judgement. A new feature is that it can be customised and optimised according to the specific network environment of a school, and despite having weaker hardware, it has shown good performance in task isolation and scheduling.

Keywords: Online Judge, Microservices, Sandbox Isolation, High Concurrency, Asynchronous Processing

目　录

摘　要
ABSTRACT
1 绪　论
1.1 研究背景与意义
1.2 国内外研究现状
1.3 论文主要研究内容与目标
1.3.1 论文主要研究内容
1.3.2 论文研究目标
1.4 论文组织结构
2 相关技术及理论综述
2.1 微服务架构与 Spring Cloud 体系
2.1.1 微服务架构理论基础
2.1.2 Spring Cloud Technology Ecosystem
2.1.3 基于微服务的判题分发模型
2.2 容器化沙箱技术与 Linux 内核隔离机制
2.2.1 基于 Linux 内核的原生隔离机制
2.2.2 go-judge 的技术优势与特性
2.3 前端响应式开发框架 Vue 3
2.3.1 Vue 3 与 TypeScript 的深度融合
2.3.2 基于 PrimeVue 4 的 UI 组件实践
2.3.3 Axios 请求库的底层封装与异常处理
2.3.4 多层级路由架构设计
2.4 分布式中间件（Redis、RabbitMQ）原理
2.4.1 Redis 高性能缓存与分布式锁
2.4.2 RabbitMQ 消息队列与异步分发原理
2.4.3 中间件协同下的高并发治理逻辑
3 系统分析与总体设计
3.1 系统可行性分析
3.1.1 技术可行性
3.1.2 经济可行性

3.1.3 操作可行性
3.1.4 法律与社会可行性
3.2 系统功能性需求分析
3.2.1 用户与反馈功能
3.2.2 题目管理与展示模块
3.2.3 代码评测与调试模块
3.2.4 比赛与作业组织模块
3.3 非功能性需求分析
3.3.1 性能与吞吐量需求
3.3.2 可靠性与可用性需求
3.3.3 系统安全性需求
3.3.4 可扩展性与维护性需求
3.3.5 兼容性与易用性需求
4 系统总体架构设计
4.1 逻辑结构设计
4.2 前端工程化设计
4.2.1 模块化路由拆分方案
4.2.2 Axios 请求库二次封装
4.3 后端工程化设计
4.3.1 微服务模块划分
4.3.2 业务微服务分层架构
4.3.3 授权认证微服务架构
4.3.4 判题机微服务架构
4.3.5 微服务缓存策略
4.4 系统数据库结构设计
4.4.1 数据库设计原则
4.4.2 系统核心实体关系设计
4.4.3 用户与权限模块数据设计
4.4.4 题目与比赛模块数据设计
4.4.5 用户与权限模块数据设计
5 系统核心功能的实现
5.1 用户认证与授权功能实现
5.1.1 认证请求处理流程
5.1.2 登录成功和失败处理流程
5.1.3 验证码校验流程实现

5.2 分布式判题功能实现
5.2.1 提交任务与异步调度
5.2.2 判题节点执行
5.2.3 测试用例与题目数据获取
5.2.4 安全沙箱执行（Sandbox）
5.2.5 特殊判题和交互判题处理
5.2.6 判题结果回写与统计更新
5.3 比赛与排行榜功能的优化实现
5.3.1 研究背景与问题分析
5.3.2 总体优化思路
5.3.3 关键实现设计
7 系统测试与实战分析
7.1 测试环境与测试方案
7.2 系统功能测试
7.2.1 用户认证功能测试
7.2.2 评测功能测试
7.2.3 比赛与排行榜功能测试
7.3 分布式判题压力测试
7.3.1 高并发提交测试
7.3.2 RabbitMQ 异步调度分析
7.3.3 Sandbox 隔离执行测试
7.4 缓存与排行榜优化效果分析
7.4.1 Redis 缓存优化效果分析
7.4.2 分布式锁与缓存一致性分析
7.5 实战比赛运行分析
7.5.1 比赛运行情况
7.5.2 系统运行效果分析
7.6 本章小结
8 总结与展望
8.1 总结
8.2 展望
结论
参考文献
致谢

1 绪论

1.1 研究背景与意义

伴随着计算机学科竞赛在高校教育中越来越重要，在线评测系统（Online Judge）也成了提高学生算法能力和工程实践的主要手段。但是济南大学原来的评测系统由于存在单体架构，面对大规模的校级赛事时就暴露出高并发承载力不够、响应时间长、判题环境不安全等众多问题。除此之外，校园网多层代理、WebVPN 访问等使得系统部署以及网络通信变得比较困难。因此，在这样一种情况下，采用现代微服务架构以及 Linux 内核级隔离技术，从零开始建立一个能够适应复杂网络环境、能够支撑高并发业务并具有工业级稳定性的评测平台，就是学校信息化教学建设的迫切需要。

本文用 57000 行核心逻辑代码的工程实践来证明，单体架构向分布式架构转变之后，系统工程的价值得到了极大的提高。系统加入了 cgroups、Namespace 来保证判题沙箱的安全性以及资源公平分配，用 RabbitMQ 异步分发成功解决了瞬间高压时性能瓶颈的问题。从实际使用情况来看，新系统平滑地迁移到了全部的历史数据上，并且可以稳定地支撑起每天超过 4000 次的单个比赛的提交工作，峰值达到了 350QPS。该项目成功完成以后，不仅给校内程序设计竞赛构建起了稳固的技术架构而且给个人高效地创建起高可靠、高并发的大规模软件工程赋予了可以借鉴的范例。

1.2 国内外研究现状

在线评测（Online Judge，OJ）系统在国际上起步较早，也逐渐形成了比较成熟的工程体系。典型的平台有西班牙的 UVaOnlineJudge、波兰的 SPOJ 等，它们一直为程序设计竞赛的训练和学习提供着算法学习环境。随着用户数量的增加以及应用场景的扩大，国外的在线评测系统也由原来的“判题功能实现”变成了现在的综合服务平台，具有较高的可用性和可扩展性。

近些年来，云计算以及虚拟化技术的发展使得在线评测系统在安全性以及资源隔离方面取得了很大的进展。由于判题任务一般要执行用户提交的代码，存在恶意行为的风险，所以怎样创建安全沙箱成了研究的重点。目前比较常见的是用 Linux 容器技术（Docker）加上 cgroups、namespace 等机制来达到进程级资源限制、运行环境隔离和权限控制的目的，进而提高评测系统安全性和稳定性。而且一些平台又加入了日志审计、系统调用过滤这些手段来加强对于恶意程序的防护。

除了 LeetCode、HackerRank 等商业化的在线编程平台以外，其他一些在线评测系统，比如在处理高并发问题时有着突出表现的，有评测响应速度的，还有具备交互题、特判题、在线运行调试等能力的系统。该类平台一般会用到分布式架构、异步任务调度和多级缓存等手段来应对大量的同时访问。

在国内，线上评测系统的产生和程序设计竞赛、高校算法教学的开展有关。早期的平台，如北京大学的 POJ 等，在算法训练方面有着重要的影响，其系统设计大

多采用单体架构，可以满足早期竞赛训练的需求，但是弹性扩展、运维自动化、高并发支持等方面存在一定的不足。随着容器化、云原生理念的普及，国内也出现了许多开源的在线评测系统，如 QingdaoU Online Judge 等项目使用 Docker 容器来部署，提高了环境的一致性以及系统的可维护性，也促进了高校和社区对于现代判题架构的研究。

目前我国关于在线评测系统学术研究及应用的热点，主要集中在判题服务模块化、微服务化改造、评测任务异步调度和队列管理、判题环境安全隔离、面向教学场景功能扩展等方面。消息队列可以用来做判题任务的削峰填谷，多节点评测集群可以提高吞吐量，已经成为比较常见的系统优化途径。但是从工程应用角度来说，在高校内网环境、WebVPN 映射、多层代理访问等复杂的网络环境下，系统适配的问题还没有形成统一的、成熟的通用方案，大部分都是根据具体的部署场景来定制化的进行设计和优化的。

1.3 论文主要研究内容与目标

1.3.1 论文主要研究内容

本文根据济南大学在线评测系统重构的需求，从架构设计、核心开发、工程运维三个方面对济南大学在线评测系统进行研究。首先对 Spring Cloud 生态体系下的高性能微服务架构进行研究并实现，通过业务逻辑的解耦将原来的单体系统分解成多

个独立的服务，并且使用 Nacos 服务发现和配置中心来确定系统是分布式的。其次就是对安全判题机制进行研究，用 Linux 内核级别的 cgroups 和 Namespace 技术对判题沙箱做了强隔离，并开发了支持多语言编译、特判(SPJ)和交互判题的调度算法。在并发处理上主要对用 RabbitMQ 消息队列实现异步任务分发系统、用 Redis 分布式锁来解决高并发下数据的一致性以及缓存击穿问题做了研究。除此之外，对于校园网内特殊的环境下使用 Nginx 反向代理协议来解决跨域通信问题，在 WebVPN 环境下也对 Nginx 反向代理做了更加深入的研究。最后利用 GitHub 进行工程化的版本控制，对核心逻辑代码进行了 5.7 万行代码的编写和审计，并设计出无损迁移旧系统数据的方法，保证新旧平台的平滑过渡。

1.3.2 论文研究目标

本课题意在利用全部的软件工程操作来创建出一个稳定的、安全的、具有工业级性能的数字化竞赛支持系统。第一项主要任务就是彻底消除旧版系统在大型赛事中出现的响应卡顿、宕机等问题，使得单日判题量大于 4000 次、峰值请求量大于 350QPS 的稳定运行，把平均判题延迟明显降低 60%以上。从安全性来说，要创建起完备的判题沙箱隔离体系，保证非法代码不会对宿主机的安全造成影响，而且还要达成核心业务逻辑的零越权访问。除此之外，系统还应具有应对复杂校园网访问模式的能力，在校园内和校园外都能够获得良好的使用效果。最终经过 5.7 万行高质

量代码的积累和第八届新生赛的实战检验，本文主要目的在于创建起可以借鉴的高校自主重构大型教学评价系统工程范式，给学校的计算机专业教学及选拔赋予稳定的技术支撑。

1.4 论文组织结构

本论文共分为七章，各章节内容安排如下：

第一章：绪论。本章主要对在线评测系统的背景和实际意义进行了阐述，在介绍了国内外 OJ 系统及判题技术研究现状的基础上，提出了本文的研究内容和最终实现的目标。

第二章：相关技术及理论综述。本章对系统构建所用到的主要技术做了详细的调研。微服务架构、Spring Cloud 体系、Nacos 服务治理方案、前端 Vue3 框架、中间件（Redis、RabbitMQ）原理、判题沙箱核心底层技术Linux cgroups、Namespace 隔离机制。

第三章：系统分析与总体设计。本章根据实际需要，对系统的功能需求和非功能需求做了详细的分析。在此基础上对系统进行了微服务模块划分、总体技术架构和数据库的核心实体关系模型（E-R 图）。

第四章：核心功能模块的实现。本章对系统开发过程做了详细的介绍。本文主要讨论了现代化交互界面的创建、分布式判题引擎的调度方式、依靠消息队列的异步任务处理系统、高性能安全沙箱的接入和封装。

第六章：系统测试与实战分析。本章对系统做了全面的测试。对功能模块测试、350QPS 高并发压力测试以及第八届程序设计新生赛期间真实的运行数据进行分析，从而证明系统的稳定性和可靠性。

第七章：总结与展望。本章对本文的主要工作成果和创新点进行总结，客观分析了系统目前存在的不足之处，并对未来的改进方向和功能扩展做了展望。

2 相关技术及理论综述

2.1 微服务架构与 Spring Cloud 体系

本系统重新设计时使用的是微服务架构，它是高性能、高可用的理论基础。

2.1.1 微服务架构理论基础

微服务架构是把一个应用程序拆分成一组小的服务，从而形成的一种软件开发方式。每一个服务都是独立运行在不同的进程中，用轻量级通信方式（HTTP RESTful API 或者 gRPC）来交互。相比于传统的单体架构（Monolithic），它的主要优点在于具有以下几点。

解耦、独立性，各个功能模块（用户中心、判题调度、题目管理）独立开发、独立部署，不会出现牵一发而动全身的情况。

水平扩展性就是对于需要高负载的判题服务模块，在实际使用中自动调整其实例的数量，以达到对资源进行有效分配的目的。

故障隔离就是对某个微服务出现的异常情况，用熔断机制来保证核心业务逻辑不会被阻塞，从而避免整个系统出现级联失效的情况。

2.1.2 Spring Cloud Technology Ecosystem

Spring Cloud 是一套完整的分布式系统解决方案，本系统使用 Spring Cloud 的全家桶组件来解决微服务治理中遇到的主要问题。

服务发现和注册，是系统中的通讯录，所有的微服务在启动的时候都会自动注

册到Nacos注册中心，服务之间通过服务名进行调用，避免了由于分布式环境而产生的动态IP地址漂移问题。

统一配置管理就是把所有的配置文件集中到一个地方进行管理，并且可以实时热更新，不需要重启服务就可以对判题频率、阈值设置等业务参数进行修改。

网关服务是系统唯一的流量入口，对所有的认证、授权、请求过滤、跨域处理以及复杂的路径映射进行处理。

2.1.3 基于微服务的判题分发模型

本系统采用Spring Cloud框架来创建任务生产、消息中间件、分布式消费的评测系统。后端业务微服务是生产者，把待判题请求封装成消息后发到消息队列里；判题微服务是消费者，从队列里取任务，调用底层的 go-judge 沙箱。该架构把业务层和判题层分离出来，在高并发的情况下仍然能够用微服务间的流控机制来保证响应的稳定性。

2.2 容器化沙箱技术与 Linux 内核隔离机制

在线评测系统中沙箱（Sandbox）的主要工作就是给不可信的用户代码提供一个受限的运行环境。抛弃了重量级的虚拟机方案，用基于Linux内核特性的小型容器隔离方案，把开源高性能判题沙箱go-judge作为主要的判题引擎。

2.2.1 基于 Linux 内核的原生隔离机制

go-judge的核心安全特性是依靠Linux内核提供的原生隔离能力来实现的，主

要包含以下几个方面。

（1）Namespace 隔离：通过 CLONE_NEWNS、CLONE_NEWPID、CLONE_NEWNET 等标志位，为每一个判题任务创建完全隔离的文件系统、进程树和网络环境。这就使得用户代码不能感知宿主机进程，在逻辑上是断网的，从根本上杜绝了网络攻击的可能性。

（2）Cgroups 资源限制：利用 Linux cgroups 实时监控并限制判题进程的 CPU 时间片消耗、物理内存使用量以及最大进程数（Task Limit）。相比传统监控方式，cgroups 能够在内核层级精准拦截内存溢出（OOM）或 Fork 炸弹攻击。

（3）Seccomp 指令过滤：这是 go-judge 提升安全性的关键。通过配置 BPF 规则来过滤系统调用，禁止用户程序调用 mount、ptrace、socket 等危险的系统调用。即使攻击者得到了沙箱里的 root 权限，也不能对宿主机的内核造成实质性的损害。

2.2.2 go-judge 的技术优势与特性

本文选择 go-judge 而不是传统的 Docker 容器，主要是因为 go-judge 在 OJ 场景下进行了深度优化。

非持久化容器模型和 Docker 的守护进程不一样，go-judge 给短生命周期的判题任务设计了任务即即毁的模式，从而减少大量的容器创建、销毁引起的 I/O 开销。

（2）基于内存的文件系统，go-judge 在 tmpfs 上进行编译和运行，减少磁盘 I/O 对判题速度的影响，在本系统实测的 350 QPS 下有明显的优势。

（3）采用 RESTful/gRPC 的异步通信方式同 go-judge 进行通信，使用 RabbitMQ 来实现业务逻辑和底层判题环境的解耦。它自带的流式文件传输功能大大提高了大数据量测试点的传输速度。

2.3 前端响应式开发框架 Vue 3

本项目使用Vue3作为前端主要框架，配合TypeScript、PrimeVue4和自定义化的结构来创建一个包含22000行高质量的非配置代码的项目。

2.3.1 Vue 3 与 TypeScript 的深度融合

Vue 3 的 Composition API 为本项目的大规模逻辑复用提供了原生支持，而 TypeScript 的引入则为 2.2 万行代码的健壮性提供了工程底气。用严格的类型接口（Interfaces）来定义题目元数据、提交状态、竞赛配置等主要业务对象，对系统所有业务对象都进行了类型约束。强类型开发模式一方面在编译时就拦住了大多数因为字段空缺或者类型不兼容而引起的运行时错误，另一方面也用准确的代码补全大大提高了重构效率，在分布式架构的环境下使得前端的数据模型和后端的DTO保持一致。

2.3.2 基于 PrimeVue 4 的 UI 组件实践

本项目选择了PrimeVue 4为底层的UI组件库，使用它的全新的非受控样式系统以及响应式布局。为了解决在线评测系统特有的高密度数据展示问题，本文在PrimeVue的基础上做了深度定制，开发出实时倒计时组件和多维数据统计图表。相比传统的UI框架，PrimeVue4轻量化的组件逻辑使得前端只需要小量的性能开销就可以完成大规模的DOM更新，在保证视觉统一性的基础上，在PC端和WebVPN环境

下都能达到很好的兼容效果。

2.3.3 Axios 请求库的底层封装与异常处理

为了规范管理 5.7 万行代码量下的前后端异步交互，本项目对 Axios 做了工业级的二次封装。创建全局拦截器体系之后，就达成了 JWT 自动注入，请求防抖以及响应数据格式的预处理。异常处理上采用了分层的熔断反馈机制，将业务层异常（题目权限不够等），网络层异常（500 错误等），认证失败异常（Token 失效等）分别进行捕捉，并用 PrimeVue 消息通知组件做了非侵入式的用户提醒。封装模式把各个业务模块的逻辑代码都封装起来，从而达到前端请求层高度解耦、可复用的目的。

2.3.4 多层级路由架构设计

本项目用 Vue Router 实现复杂多层级嵌套路由结构的 OJ 系统。将前台门户、用户个人中心、竞赛竞技场、后台管理系统分别作为独立的路由模块，使得各个业务模块的路由定义可以独立编写，并且可以根据需要汇总。该种设计模式一方面解决了一个人开发时路由配置文件过多难以维护的问题，另一方面又利用了路由守卫做了模块级的访问控制。高度模块化路由组织形式可以使系统横向扩展功能的时候，仍然能够使代码组织结构清晰，逻辑运转顺畅。

2.4 分布式中间件（Redis、RabbitMQ）原理

在分布式架构当中，中间件起到了解耦、异步、削峰的作用。本项目通过使用Redis、RabbitMQ来解决后面57000行代码之后的数据一致性以及高并发调度的问题。

2.4.1 Redis 高性能缓存与分布式锁

Redis 是一种基于内存的非关系型数据库，本系统里它起到了高频热点数据存储以及并发冲突控制的作用。其主要价值就是使用单线程多路复用的方式达到万级以上的QPS，系统把实时的题目元数据、用户的Session、动态排行榜存放在内存里，大大减轻了对MySQL数据库的直接访问压力。工程实践上用Redis的原子性指令实现了分布式锁（Setnx）来保证微服务集群环境下的任务抢占、报名计数这些业务不会出现数据竞争和逻辑覆盖问题。瞬时流量冲击为350QPS的时候，系统会执行比较细致的缓存失效和预热策略来避免由于高并发比赛导致的缓存击穿问题。

2.4.2 RabbitMQ 消息队列与异步分发原理

RabbitMQ 是一款基于AMQP协议的工业级消息中间件，它把后端业务逻辑和底层的go-judge进行连接。系统依靠它的“生产者-消费者”模式把判题流程彻底变成异步的，即后端微服务接收到用户提交的代码之后，并不会立刻调用判题逻辑，而是会将任务打包成标准的消息投递到RabbitMQ交换机上。该种架构设计把昂贵的CPU 密集型判题任务和轻量级的 Web 业务请求解耦，而且借助队列的缓冲作用来实现削峰填谷的效果，在新生赛这样的瞬间性压力情况下，系统可以把海量的请求变

成比较平缓的任务流。开启消息持久化以及手动应答(ACK)，在网络出现异常或者某个判题节点突然出现问题的时候，用户的提交数据仍然可以追回到结果中，达到最终一致性。

2.4.3 中间件协同下的高并发治理逻辑

在本项目中，Redis 和 RabbitMQ 一起形成一个功能互补的闭环。Redis 进行状态极速优化，用户提交频率、判题机心跳检测、分数累加等都是用 Redis 加速完成的，保证用户的互动是及时的；RabbitMQ 负责业务的有条理地编排，使用有序调度、重试来保证业务链路的稳定。双中间件架构相结合之后，就成为了支撑系统在第八届程序设计新生赛中，单日处理超过 4000 次提交，并且没有出现过任务丢失或者数据库死锁等底层工程的底座。

3 系统分析与总体设计

3.1 系统可行性分析

本文就济南大学在线评测系统重构工作从技术可行性、经济可行性、操作可行性、法律可行性这四个角度来对项目可行性做分析，保证项目可行。

3.1.1 技术可行性

本文所用的微服务架构和容器化沙箱技术在国内国外已经有大量的工程案例。后端基于 Spring Cloud 体系，利用其提供的 Nacos 服务治理、OpenFeign 服务调用及 Spring Cloud Gateway 网关技术，能够有效解决单体系统的扩展性难题。底层判题引擎采用开源项目 go-judge，go-judge 利用 Linux 内核本身的 cgroups、Namespace 机制来达到安全隔离的效果，技术方案既有理论深度又有良好的性能表现，可以满足 350QPS 的实战要求。前端通过 Vue 3、TypeScript 与 PrimeVue 4 的组合，配合二次封装的 Axios 请求库和多层路由架构，能够支撑 22,000 行逻辑代码的高质量交付。因此，本项目所用的技术栈成熟，技术方案可行。

3.1.2 经济可行性

本系统属于校园信息化基础设施的重新构建项目，它的经济性主要是由于资源的优化和维护成本的降低所体现出来的。首先，系统采用开源技术栈（如 Redis、RabbitMQ、MySQL 及 go-judge），无需支付昂贵的商业软件授权费用。其次，采用

微服务化改造和容器化部署之后，系统对于硬件资源的利用率得到了很大的提高，可以运行在学校的现有服务器集群上，不需要另外购买高性能的物理设备。最后系统用 5.7 万行高质量代码的积累和自动运维的设计大大减少了后期的人工维护成本以及系统宕机造成教学损失。所以项目在投入产出比上具有很高的经济可行性。

3.1.3 操作可行性

系统一开始就考虑到了用户的使用习惯和校园网环境的特点。前端界面使用 PrimeVue 4 进行了深度定制，可以给师生提供一种响应式的交互体验，PC 端和移动端可以无缝切换。为了使校园网内特有的 WebVPN 访问机制得以实现，系统对 Nginx 的路径映射以及跨域策略进行了相应的优化。管理后台结构清晰，可以批量导入题目，也可以设置用户权限，可以随时查看比赛，大大提高了管理员的操作效率。系统还承担过第八届程序设计新生赛的组织工作，在使用该系统的过程中检验它的业务场景下操作简便、稳定的特点。

3.1.4 法律与社会可行性

本项目属于自主研发的教育教学辅助平台，没有造成商业侵权的行为。系统引用的开源项目（如 go-judge、Spring Cloud 等）均遵循相应的开源协议。就数据安全而言，系统利用业务逻辑审计来修复 BOLA 越权访问的主要漏洞，有效地保护了参赛者的个人信息、成绩数据等，完全符合有关信息安全和数据保护的法律法规。

系统的创建可以明显提高学校的程序设计教学水平，给选拔拔尖创新人才提供技术支持，有较好的社会效益。

3.2 系统功能性需求分析

根据原 HUSTOJ 系统存在的问题以及济南大学教学环境的实际情况，从系统的各个方面进行了重新的设计。

3.2.1 用户与反馈功能

旧系统平滑登录，无新系统账号的用户可以凭旧 OJ 账号进行登录，系统会自动创建账号并同步账号信息。

个人主页管理功能可以查看用户的个人统计信息，即总提交题次数、总解决题次数、全站排名等，并且可以对用户的基本信息和密码进行修改。

多模式账号导入使用点名册格式批量导入学生信息，去掉前面五行多余数据，可以在导入之前查看导入结果并及时修改。

批量账号生成可以一键产生具有某种前缀的 ACM 竞赛临时账号。

生命周期控制管理员可以对全局用户进行修改，也就是修改用户的昵称、密码、账号过期时间以及禁止使用的用户等。

登录限流，第一次和第二次登录不用验证码，第三次开始使用验证码，同一个 IP 每次只能尝试登录 10 次。

异地登录受限制，管理员设置 IP 限制的话，用户只能在机房范围内（IP 范围）

登录系统。

登录 IP 限制策略，对某个用户或者一群人都可以设置成登录 IP 白名单的形式，可以使用 IPv4 或者 IPv6 的格式，支持批量导入，同时还可以覆盖配置。

过期访问拦截就是设置账号有效期，系统会自动禁止已经过期的登录。

站内信私聊是用户使用系统内置的信件功能与其它用户进行一对一交流的方式。

用户对系统的功能模块进行 BUG 反馈或者新的功能要求的提出以及其它问题的时候可以在线上进行反馈。

待完成任务提醒，在作业或者比赛结束之前（剩余 6 小时）会收到系统自动发送的站内信提醒，显示还没有完成的题目数量。

反馈响应治理就是管理员在管理界面集中查看、处理用户反馈意见的一种方式，可以发布全局或者特定比赛的补充通知。

消息自动发送，根据系统设置进行执行，即对需要催缴的用户进行提醒，通过邮件的形式向用户发送。

3.2.2 题目管理与展示模块

题目列表展示，全局题目列表，显示题目 ID、题目标题、总提交数、AC 次数、平均通过率、个人通过状态。

多维搜索检索就是利用题目名称、题目标签或者题库进行题目快速筛选的方式。

多组样例支撑，即对同一个题目可以设置无限个输入输出样例。

题目详情获取就是查看 Markdown 或者 HTML 格式的题面描述、输入输出规范、多组样例测试点和时间/空间限制。

Markdown 标准支持：全面支持 Markdown 编辑与渲染，同时兼容旧系统的 HTML 文本显示。

多维数据统计，题目列表即时显示 AC 率、各个判题状态（WA、TLE、RE 等）所占比例、题标的标注以及个人通过的状态。

全功能题目编辑器可以对题目进行增删改查，并且有 Markdown 编辑环境和标签分类管理。

测试点精细化治理功能可以上传测试点压缩包，单文件增加删除、测试点重命名以及设置遇错停判或者全部评测的方式。

高级判题配置包含编写、上传、在线编译特殊的判题程序 SPJ 以及交互式判题程序。

题目外部导入，支持 FPS 格式题目导入，可以从远程 OJ（HDU、POJ 等）下载题目元数据。

3.2.3 代码评测与调试模块

在线提交与语言选择：支持 C/C++（含多标准版本及 O2 优化）、Java、Python3、Go、Rust 等语言提交。

实时判题监控，提交代码之后在当前页面就可以直接看到判题进度，不需要刷

新页面或者跳转。

根据洛谷风格用颜色块直观显示各个测试点的耗时、内存使用、错误堆栈等信息。

个人历史记录，查看自己对于该题所有历史提交的代码以及详细的报错信息。

评测环境监控功能就是对各个分布式判题机（Judge Server）的工作状态以及系统负载进行实时的监控。

管理员可以调节判题任务的优先级，还可以对 Redis 缓存以及 RabbitMQ 队列的分发方式加以改进。

判题数据回溯可以查看所有的用户提交的代码，用以进行学术诚信的检查或者逻辑上的分析。

重判机制是指管理员可以对比赛内指定的题目或者指定的提交记录进行重新评测，自动更新判题结果和排名数据，保证评测的准确性、公平性。

3.2.4 比赛与作业组织模块

赛前信息预热，比赛开始前用户可以浏览比赛首页的公告、赛制规则、题目数量等内容做足准备。

在线提问系统，用户可以对题目理解有误的地方提出问题，马上查看到答疑回复

题目列表和状态显示，清楚地显示比赛题目索引、个人通过情况以及该题在本

次比赛中总通过率。

实时更新排行榜，即根据提交结果马上算出每个队员的排名和罚时。

定位到我，也就是大规模竞赛中快速定位选手的位置，使选手能够在众多榜单中迅速找到自己的位置。

判题流监控就是实时查看全场提交的判题状态（AC、WA、TLE、RE 等），绘制判题状态分布图。

提交频率分析就是用时间轴图来呈现各个时间段的答题热度，从而发现学生答题的节奏。

数据清洗与维护，去掉排行榜中管理人员的投票记录，前台排行是真实公正的。

排行榜深度导出，可以导出全部成绩的 Excel 文件，也可以根据教学需要筛选出至少做了一道题或者没有做过的学生的名单。

题目过题统计就是对每一道题目通过率和错误分布进行分析，用以课后讲评以及难度校准。

可视化选题和搜索，抛弃了传统的手动输入题目 ID 的方式，用题目标题、标签、题库来源等可以实现可视化检索，并且可以一键添加到比赛当中。

协助用户模式，在加入了新的协助用户功能之后，可以让主讲教师把比赛发布、公告发布、答疑权限分发给其他的管理员。

可见性控制，设置比赛公开、私有（凭密码入）、只允许指定班级入。

班级化一键选中，设置好班级功能之后就可以将目标学生群直接关联起来，而不需要每次都去手动导入名单。

点名册和临时账号，可以批量导入参赛者到点名册中，也可以生成带有ACM_2026_前缀的临时竞赛账号。

赛制和判题策略预设，可以设置 ACM 赛制，也可以为某个题目设置“遇错停判”或者“全部评测”的模式。

3.3 非功能性需求分析

非功能性需求指系统在运行时要达到的标准，是保证 57000 行核心代码在学校的高并发环境下可以稳定工作的基础。

3.3.1 性能与吞吐量需求

系统可以承受 500～1000 个并发用户的冲击，比赛开始时突然出现大量数据流量也可以得到很好的处理。

判题响应时延是指系统处于正常工作状态时，用户提交的代码被系统接收并开始排队的时间应该小于 100ms。

高吞吐量架构就是用 RabbitMQ 队列来缓存任务，在高频提交的时候不会因为数据库 I/O 瓶颈导致前端页面卡死。

秒级评测体验，简单题目总判题耗时（不包含代码实际运行时间）在秒级以内。

3.3.2 可靠性与可用性需求

服务高可用性，系统应该达到 99.9%的可用性，在教学考试和正式竞赛期间不能中断。

判题逻辑的准确性要达到绝对的程度，对于超时（TLE）、超内存（MLE）以及各种运行时错误的检测要准确无误。

异常恢复机制，即遇到某个判题实例因为代码出错或者系统故障而崩溃的时候，调度中心应该可以自动地找到它并且重新分配给其他任务。

3.3.3 系统安全性需求

内核级评测隔离就是要求代码执行只能在有限的沙箱环境里运行，Linux 内核会阻止用户的网络连接和危险系统调用等行为，从而防止用户恶意提交恶意代码来攻击宿主机。

精细化访问控制，可以基于角色的权限控制（RBAC），教师只能管理自己所参加的比赛，也可以基于 IP 地址进行登录限制来防止作弊。

敏感数据保护，用户信息和密码要被加密保存，防止数据库被泄露以后，用户的隐私得不到有效的保护。

3.3.4 可扩展性与维护性需求

微服务水平扩容，系统架构要能随着负载的变化而自动地增加或者减少判题节点（Judge Server）或者 API 业务实例。

编程语言的扩展性，就是在不改变核心逻辑的前提下，用配置文件的方式快速添加 Rust、Go 等新的编程语言支持。

多实例部署使用微服务架构来设计，可以同时部署多个模块实例来分担压力。

3.3.5 兼容性与易用性需求

旧系统数据兼容性，系统要完美地继承和适应旧版 HUSTOJ 的数据，能够正确地渲染旧版 HTML 格式的题面，支持用户使用旧的凭据进行平滑登录。

反馈结果可视化，评测结果应该用颜色块（参照洛谷风格）来展示，降低学生调试的门槛和认知的负担。

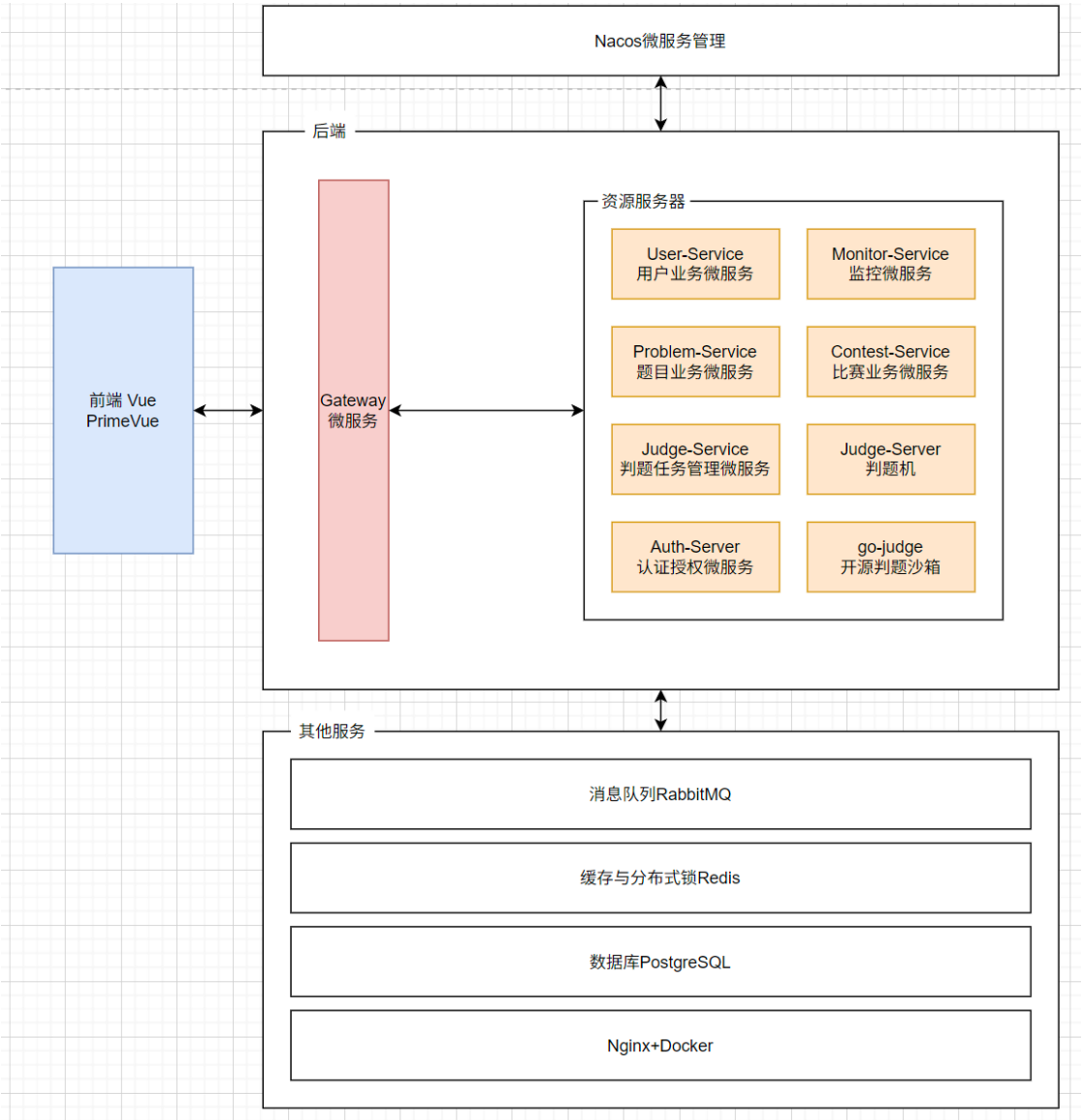
界面设计简洁大方、美观，重视交互逻辑以及功能的易用性来提高用户体验和工作效率。3.4 系统总体结构设计

4 系统总体架构设计

本系统是济南大学在线评测系统的新版本，有 57000 行后端非配置代码和 22000 行前端业务逻辑。为了满足系统需求分析中提出的需求为高并发、低延迟、高安全性的要求，所以抛弃了传统的单体结构，转而采用了基于微服务架构和前后端分离的方式构建了一个分布式工程拓扑。

4.1 逻辑结构设计

本系统采用微服务架构，使用 Spring Cloud Alibaba 生态做服务治理，把业务逻辑和判题引擎分开。



前端采用 Nginx 反向代理、部署及优化的方式对系统进行优化，从而保证了多个项目的共存。使用模块化路由拆分技术，把 22000 行前端代码分成各个独立的模块，并且对 axios 请求库做深入的二次封装，从而达到全局统一的鉴权、异常处理的目的。

网关层：基于 Spring Cloud Gateway，作为统一的流量入口。负责服务动态路由、限流防刷，在网关层解析客户端携带的 JWT 令牌并进行限制。

业务服务层：由 Spring Cloud 治理的微服务集群组成。各个微服务（User、Problem、Contest、Judge、Auth）职责单一，独立开发与部署，服务间通过 OpenFeign 进行高性能同步调用，使用 RabbitMQ 进行异步调用。

数据层：MySQL 存储关系型核心业务数据。

中间件层用 RabbitMQ 来做判题任务的异步调度中心，用 Redis 来做热点数据缓存和并发控制锁。

执行层：独立于业务服务的 go-judge 判题沙箱集群，利用 Linux 内核原生的 cgroups 和 Namespace 实现底层代码的安全评测。

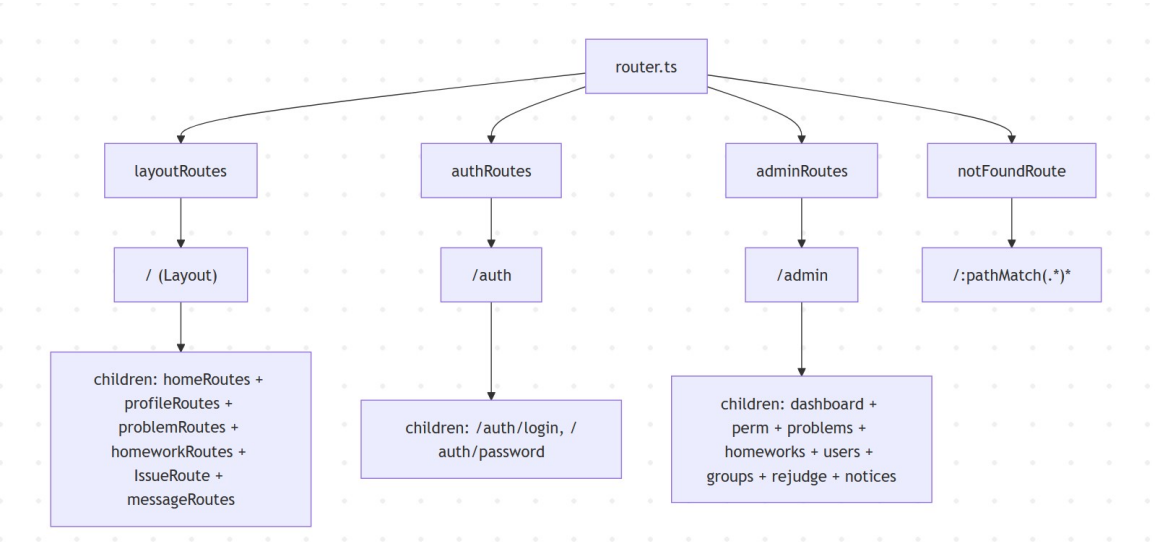
4.2 前端工程化设计

为了保证前端业务的健壮性以及减少代码的耦合度，在前端做如下两项主要的工程设计。

4.2.1 模块化路由拆分方案

由于系统功能模块很多（包含门户、判题、比赛、管理后台等），如果把所有的路由都放在一个配置文件里，就会出现并行代码提交时频繁的冲突，并且不利于大系统维护。本系统将路由按业务维度拆分为 user、problem、contest、issue、auth、message、group、 与 admin 八个独立子模块。

各个子模块分别定义路由守卫，最后由总控（router.ts）来汇集路由。该种设计可以减小系统的首屏加载时间，把前端模块分成纵向的模块。

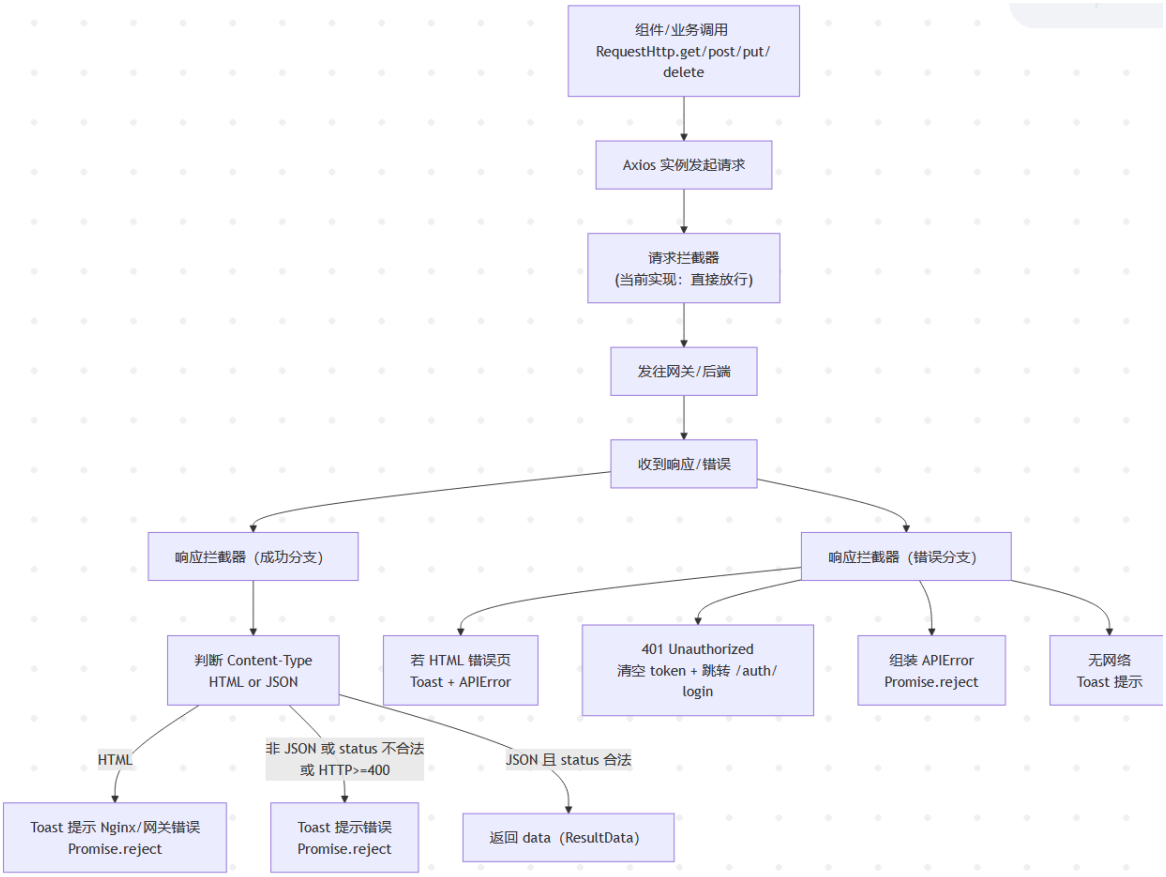


4.2.2 Axios 请求库二次封装

为了避免各个组件之间重复编写令牌注入和错误捕获代码，在底层使用了 Axios 进行封装，创建出闭环的 HTTP 交互管道

请求拦截器是从本地上下文中读取 JWT 令牌，如果存在就将其统一地放在 HTTP 请求头的 Authorization 字段里，从而达到无感知的声明式鉴权的效果。

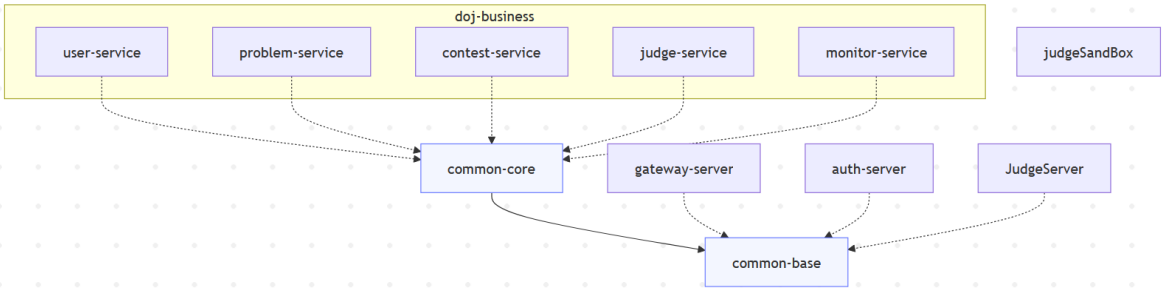
响应拦截器是对后端返回的通用数据结构 Result<T>进行拦截的。检测到业务状态码不是 200 成功状态（比如 401 登录身份过期、403 无权查看该代码、429 触发限流等）时，拦截器就会直接把后面的业务 then 链给熔断了，然后联动 PrimeVue 的 Toast 组件弹出全局错误提示，再根据状态码做路由重定向（比如跳转到登录页）。组件内部只用纯粹的成功业务逻辑来处理，大大提高了开发效率。



4.3 后端工程化设计

为了解决分布式评测系统中复杂多样的业务场景以及 57000 行核心代码庞大这个难题，后端从架构设计入手，并且做了大量工程化工作，在内部工程化结构、高并发调度、安全评测、持久化层等各方面都进行了重新设计，以提高代码的内聚性、耦合度、性能和安全性。

4.3.1 微服务模块划分



入口与治理：gateway-server 作为统一入口，负责路由、鉴权透传、限流、灰度与统一错误响应。

认证授权，auth-server 发行和校验 OAuth2/JWT；各个业务服务用公共鉴权中间件来完成鉴权和权限控制。

JudgeServer 判题机，消费判题任务，调度 Sandbox。

业务域服务（doj-business）：

user-service 是用户相关业务逻辑的提供者，可以对用户的密码进行修改，也可以实现用户增删改查。

problem-service 提供题目相关的业务逻辑，即题目信息、统计数据等。题目以及测试点的增删改查。

contest-service 是比赛相关的业务逻辑，提供比赛信息，维护排行榜，严格限制参与时间，比赛增删改查。

judge-service 判题任务创建与调度、提供提交信息。

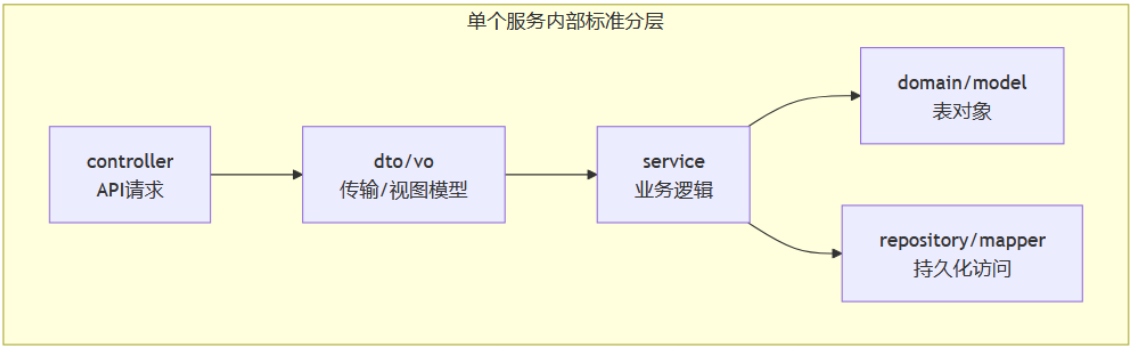
monitor-service 服务与判题监控、指标聚合与告警。

通用模块依赖（doj-common）：

common-base 通用基础常量。

common-core 业务逻辑通用配置以及公共方法。

4.3.2 业务微服务分层架构



controller 只做 API 定义、入参校验。

dto/vo 传输与视图模型，传递请求数据。

service 负责具体的业务逻辑。

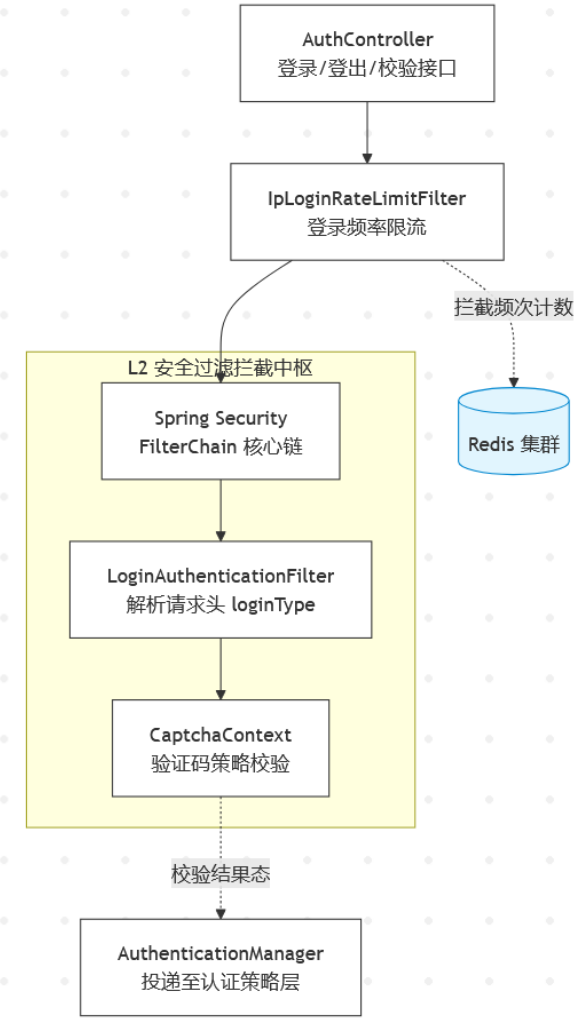
repository/mapper 负责数据库访问。

各模块严格划分 Controller、Service 与 Mapper。禁止跨层调用或者在 Controller 中编写任何业务逻辑，从而使得工程量达到几万行的时候仍然可以保持单文件逻辑短小精悍

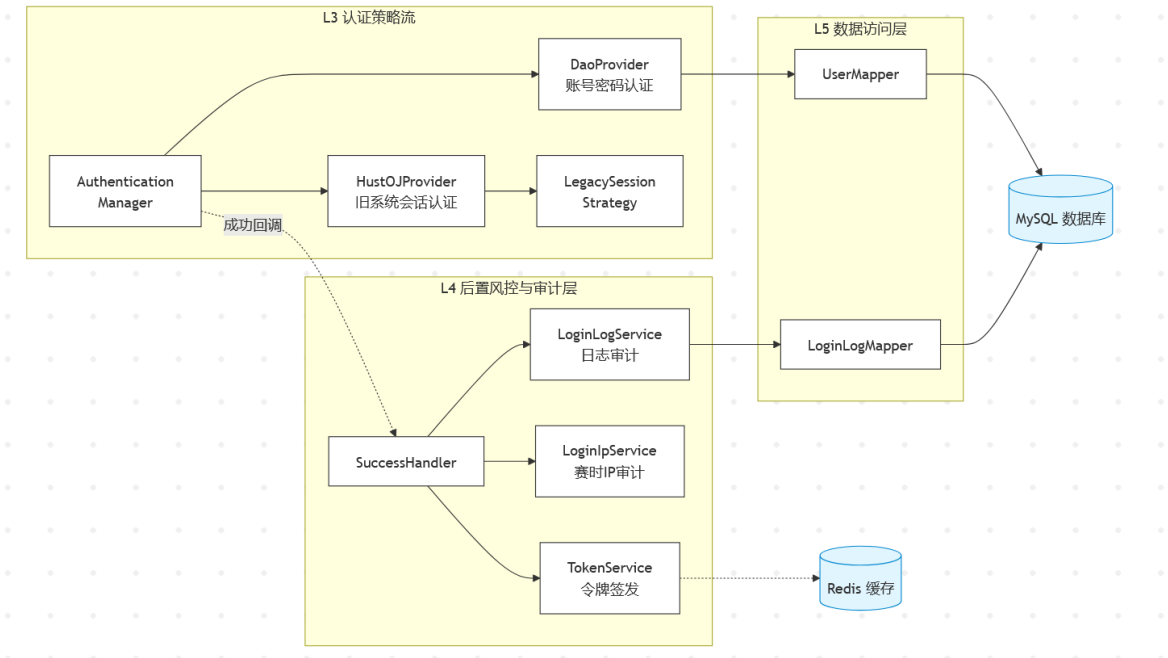
4.3.3 授权认证微服务架构

授权认证微服务属于本系统安全体系的关键部分，它主要是对用户的认证、生成访问令牌以及会话状态进行管理，给题库管理、评测服务、竞赛管理等业务微服

务赋予统一的鉴权准则。微服务架构下的认证、授权等功能被放在独立的模块里，不会造成各个业务模块之间重复写入登录校验逻辑，从而提高了系统的耦合度，也提升了系统的可维护性。系统客户端请求先经由网关服务来完成路由以及基础校验工作，登录请求会被转给认证服务去处理，而业务请求会依靠携带的 Token 来实现跨服务的身份识别和权限管控。



API 接入与安全过滤骨架图



双通道认证与后置审计依赖图

认证服务采用分层的方式对模块的功能进行组织，接口接入层、安全过滤层、认证策略层、业务服务层和数据访问层都设置了。接口层对外提供登录、登出和 Token 校验等 REST 接口；安全过滤层使用 Spring Security 构建过滤链，对验证码进行校验、对登录进行限流、捕获异常；认证策略层使用自定义 AuthenticationProvider 实现各种认证方式的可插拔扩展；业务服务层封装了 Token 生成、权限解析、失败次数统计、IP 校验等核心逻辑；数据访问层使用 MyBatis 访问用户、角色、权限、登录日志等数据表。分层结构把认证逻辑分开，便于以后添加新的登录方式和安全策略。

在会话管理上，本系统用自定义的 Token 来代替传统的服务器 Session。认证成

功之后，认证服务会生成一个随机 Token 并返回给客户端，然后把 Token 和用户标识、角色信息、权限集合以及过期时间存入 Redis，从而得到统一的登录状态存储。后续请求携带 token 的时候，网关或者业务服务可以从 redis 中直接获取用户的 id，进而实现鉴权和访问控制的高效进行。相比用数据库管理会话而言，使用 Redis 可以达到更高的并发读写性能，进而使认证与鉴权的速度得到大幅提高，并且支持 Token 自动失效以及在线用户管理等功能。

为了保证不同用户来源认证的问题，认证服务有不同的登录方式。系统根据请求头的 loginType 来区分认证流程，普通用户用用户名密码认证的方式，旧系统用户用会话标识完成身份转换和兼容登录。认证策略采用 Provider 模式实现扩展，不同登录方式对应不同 AuthenticationProvider，实现认证流程解耦。使系统可以平滑的加入新的认证方式，如短信验证码登录、第三方 OAuth 登录等，从而提高系统的扩展性。

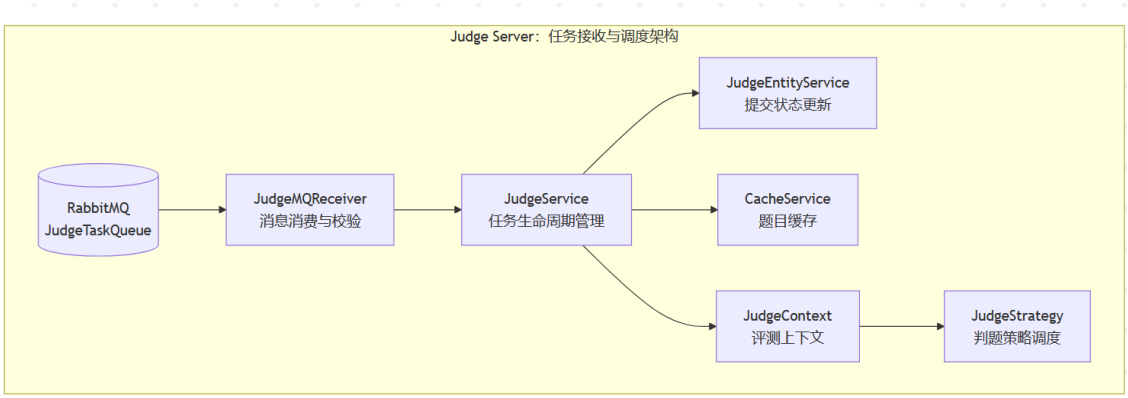
为了提高系统的整体安全性，在架构上对认证服务做了各种各样的安全防护措施，验证码校验、基于 IP 的登录限流、异常登录失败次数统计和黑名单控制等，用以减少暴力破解以及恶意请求的风险。系统会保存登录日志和审计信息，即记录用户登录的时间、登陆的 IP 地址、登陆时使用的用户名等重要信息，方便以后的安全分析、问题排查等工作。把安全控制和审计功能都集成到认证服务里，就能达成统一的管理以及策略复用的目标，从而让整个系统的安全体系更加完备。

4.3.4 判题机微服务架构

判题机微服务（Judge Service）是在线评测系统的主要计算模块，它接收评测任务、编译和运行代码、执行测试点的判断、最后把评测结果写入业务数据库。因为判题过程存在高并发、强隔离、高资源占用等特点，所以把判题功能从业务服务中分离出来，创建独立的判题微服务，采用消息队列的方式进行异步化的任务调度，从而提高整个系统的吞吐量和稳定性。业务服务在生成提交记录之后，会把评测任务封装成 ToJudgeDTO 发送到 RabbitMQ 的判题任务队列（JudgeTaskQueue），判题服务用消息监听器异步消费任务，不会让用户的请求和评测执行强绑定在一起，进而达到削峰填谷、分布式扩展的目的。

系统把判题服务分成消息接入层、业务协调层、判题上下文层、策略调度层和执行引擎层这五大部分。消息接入层使用 JudgeMQReceiver 来处理消息，主要是消费消息、校验 Token、合法性校验以及过滤掉非法的任务，保证只合法的任务进入核心判题逻辑。业务协调层由 JudgeService 来完成，它对判题任务的生命周期进行管理，即提交记录状态的更新（COMPILING、JUDGING、FINISHED），判题人的信息写入和评测结束后的结果回写，并且协调缓存模块获取题目信息和资源的分配。为了减少数据库频繁查询造成的开销，系统使用了 CacheService 这个缓存服务，把题目评测所需要的重要元数据（时间限制、内存限制、语言配置、测试用例版本等）

存入到缓存中，从而加快了判题的吞吐速度。



判题任务接收与调度架构图

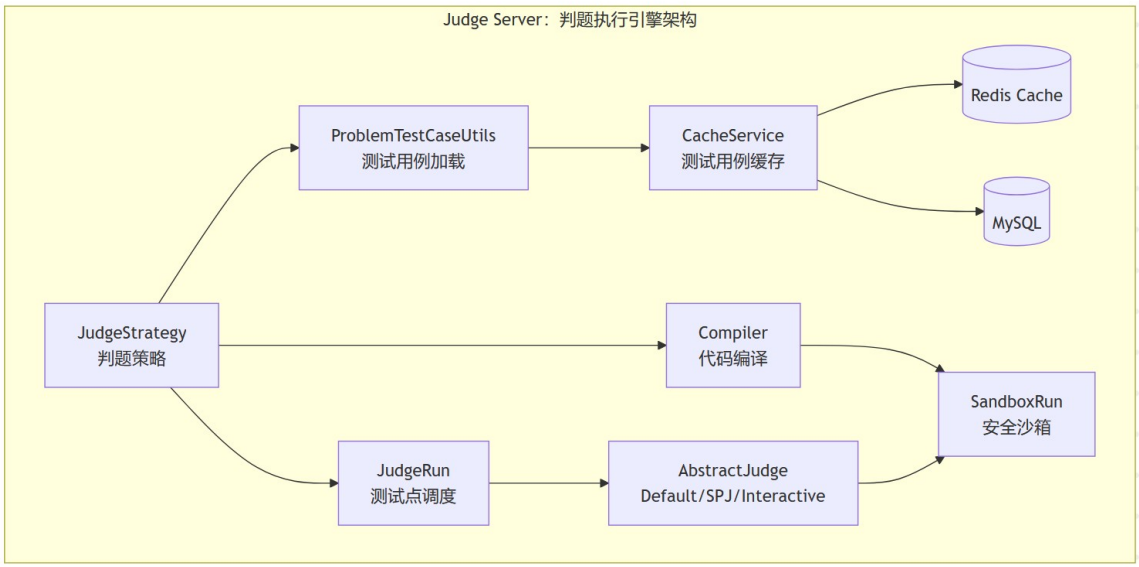


图 X-X 判题执行引擎架构图

系统中引入 JudgeContext 来充当评测上下文的容器，用来封装题目信息、提交

信息、运行环境配置以及评测过程中产生的中间状态。判题上下文向下交由 JudgeStrategy 策略模块处理，该模块用于根据题目类型选择不同的判题路径，例如普通题（Default Judge）、特判题（Special Judge）与交互题（Interactive Judge）。策略模块会调用 Compiler 对用户代码进行编译，编译任务会统一分发到安全沙箱里去执行，沙箱为用户提供隔离环境，以此来约束 CPU、内存以及系统调用，从而达到保护判题环境的目的。编译完后策略模块会调用 ProblemTestCaseUtils 的加载测试用例的方法，对测试用例的信息进行加载，在使用的时候会对特判程序或者交互程序进行预编译，使评测过程可以处理各种题型。之后系统就进入到测试点运行阶段，JudgeRun 负责调度所有的测试用例，并将执行结果汇总起来，它内部又会调用抽象判题任务模型 AbstractJudge（DefaultJudge、SpecialJudge、InteractiveJudge 等），对单个测试点进行运行和判定，最后得到一个包含分数、耗时、内存占用和判题状态的结果集。

数据一致性、任务状态控制上，由于 JudgeEntityService 是原子化的，不会因为并发任务造成重复判题或者状态的冲突。在任务执行之前系统会把状态设为 COMPILING，在更新失败的时候判定任务已经被取消或者删除，从而终止后面的流程并同步更新其他的关联表状态（提交表、比赛记录表等）。评测结束之后，把最后的结果写回数据库，并且更新提交记录的状态、分数、最大耗时这些统计字段。因

此该架构采用消息队列异步调度、策略化判题流程、沙箱隔离执行、分层模块解耦的设计方式来实现高并发环境中的稳定评测，为之后的分布式判题节点、支持多种语言运行环境、多题型判定等扩展奠定了较好的架构基础。

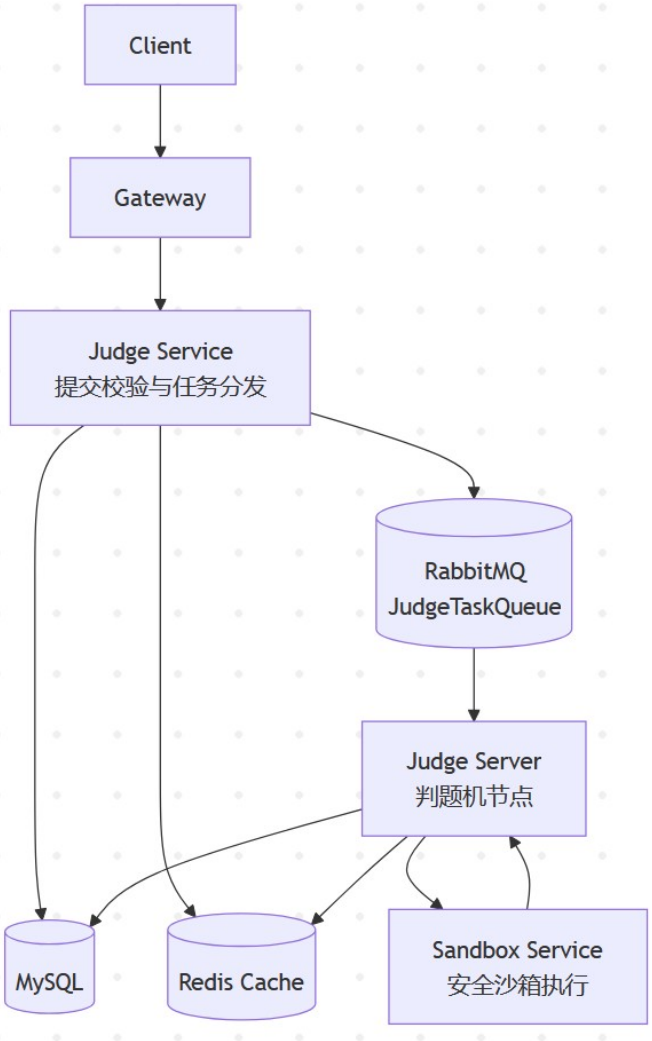
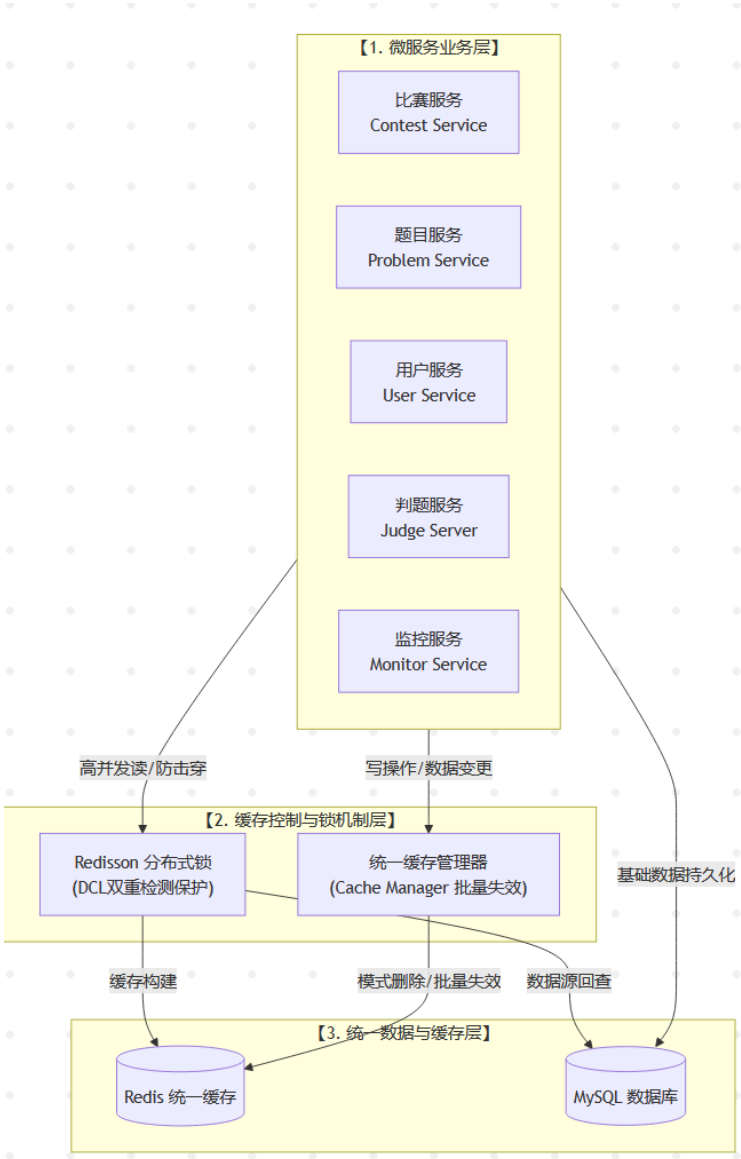


图 X-X 判题系统总体架构图（Judge Service 与 Judge Server 协同架构）

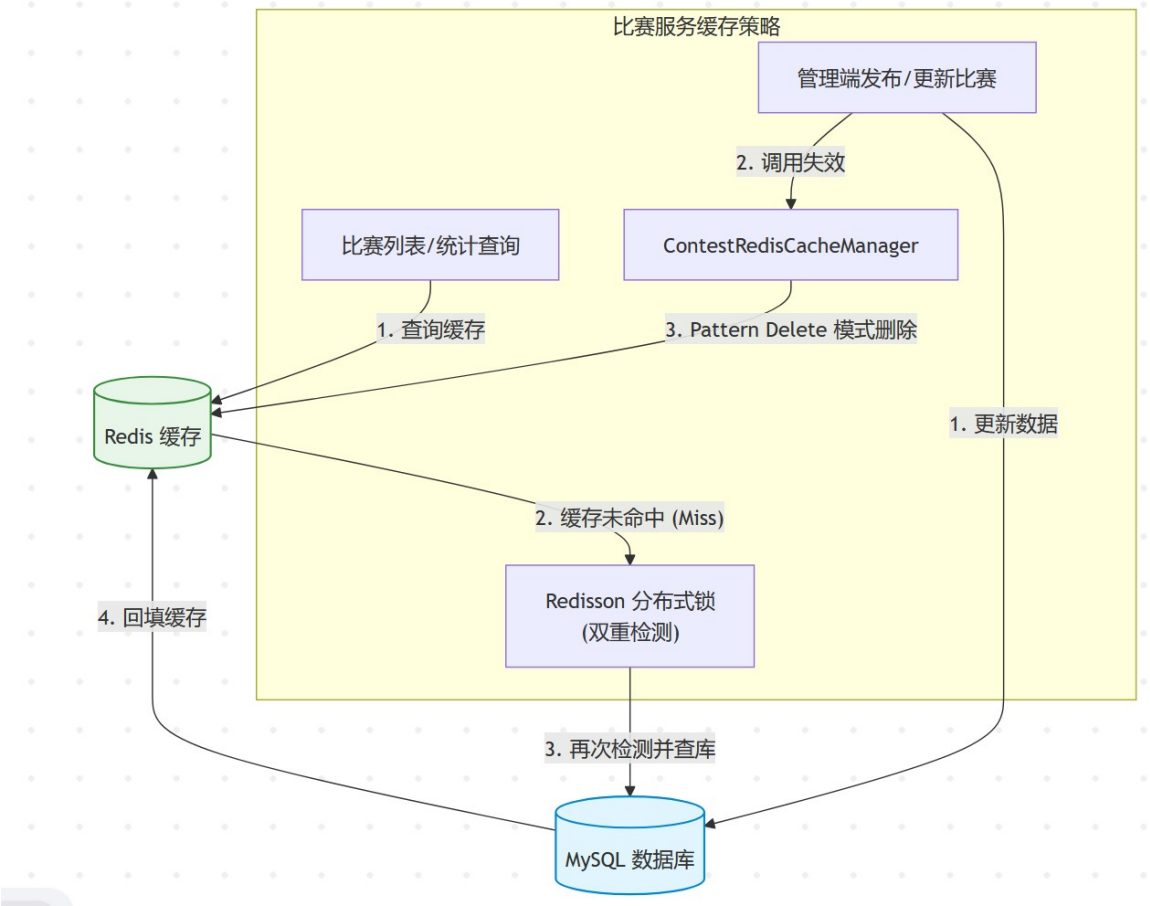
4.3.5 微服务缓存策略

为了解决高并发环境下数据库被大量调用所造成的压力以及系统反应速度慢的

问题，保证分布式环境下数据的一致性，本系统采用了统一缓存的方式进行微服务的实现。系统用 Redis 做统一的缓存中间件，用 Redisson 分布式锁机制来做热点数据保护、缓存穿透防护和高并发场景下数据一致性控制。缓存能力可以覆盖到题目服务、比赛服务、用户服务、判题服务、监控服务等各个主要微服务上，采用统一的缓存管理方式来达到各个微服务之间数据访问的优化和缓存失效的控制。



统一缓存策略总体设计



比赛服务（Contest）缓存防击穿与主动失效

缓存数据的组织方式上按照业务性质来确定缓存粒度和过期策略。对访问频率高、可以短时间的数据延迟的列表、统计信息等使用短 TTL 的 Value 缓存，而对结构化的题目详情、比赛记录等用 Hash 结构存储，并设置长的过期时间，从而减少数据库频繁查询造成的性能损失。判题模块的测试用例、题目评测信息等采用 Redis 缓存的方式来加载，在高频判题的时候可以减少数据库直接访问所造成的 I/O 压力。用户登录时的 token 以及在线状态信息会保存到 redis 里面，从而使得认证服务、监控服务进行状态查询时更快捷。

为了防止缓存雪崩、热点 Key 并发击穿的问题，系统对于某些业务使用了 Redisson 分布式锁以及缓存双检的方式。缓存未命中时，只允许获取锁成功的线程去访问数据库并回填缓存，其他的线程都要等到缓存构建完成之后再从 Redis 中获取数据。对于比赛、题目等缓存逻辑比较复杂并且需要大量 Key 管理的微服务，系统又将缓存管理器（ContestRedisCacheManager、ProblemRedisCacheManager）从业务层剥离出来，统一进行缓存 Key 命名、批量失效、模式删除等操作，从而降低业务层和缓存实现之间的耦合度，提高缓存策略的可维护性。

另外为了保证缓存和数据库之间数据的一致性，在新增、修改或者删除业务数据的时候，系统会主动调用缓存管理器来清理相关的缓存 Key，使得后面的请求重新加载最新的数据。相比传统定时刷新方式，本发明的方案可以达到更高的缓存命中率，也可以减少数据不一致时间。因此本系统采用的是 Redis 缓存、Redisson 分布式锁、缓存管理器统一缓存的方式，在高并发环境下对数据进行访问优化以及缓存一致性控制，给在线评测平台的稳定运行提供重要的性能支持。

4.4 系统数据库结构设计

4.4.1 数据库设计原则

本系统使用 MySQL 作为主要的关系型数据库，用来存放用户信息、题目数据、比赛记录、判题结果等结构化的业务数据。系统在数据库设计时按照高内聚、低耦

合、面向业务模块划分的设计思想，把各个业务模块的数据分别加以组织，从而减小模块之间数据的依赖程度，提升系统的可扩展性。

在表结构设计上系统使用了规范化的做法，并且有适当的冗余。对用户、角色、权限等基础数据用第三范式设计来减小数据冗余，对频繁查询的排名表、判题统计表等进行简单的字段冗余以降低复杂的关联查询。为了应对高并发访问的情况，在用户编号、题目编号、比赛编号、提交状态这些高频查询的字段上建立了索引，从而提高了查询速度，减小了数据库的压力。

另外为了减小高频读取操作给数据库造成的影响，系统使用了 Redis 作为缓存层来对测试用例、题目统计信息、比赛列表、用户登录状态等数据进行缓存管理。系统会把缓存中与数据库存在不一致的数据删除，用 Redisson 分布式锁、缓存双重检测的方式来防止在高并发的时候发生缓存击穿现象，以此来保证整个系统在大批量评测的时候可以一直稳定运行。

4.4.2 系统核心实体关系设计

本在线评测系统的后端数据库用组件化、高并发的表结构来设计。为了完成用户权限控制、分布式判题、竞赛排名结算、题目分类管理等主要业务，系统设计出八个主要的实体。以下是各个实体的设计、主要字段的作用和它们之间的关系。

用户实体(user_info)是系统的最基本实体，它包含了用户的账号信息，包括学号、用户名、密码、邮箱等。该实体和安全防护策略(IP 白名单、策略表)、做题记

录(user_record)有关联。

角色实体（role）是系统中定义的各个角色，即超级管理员、普通用户、比赛创建者、班级教师等，用以实现基于角色的权限控制（RBAC）。

权限实体（perm）是菜单级（MENU）和接口级（API）的细粒度权限控制规则。由于解耦了角色和权限，安全策略的平滑升级就变得容易了。

题目实体(problem)是评测系统的主体部分，保存着题目在时间、空间上的约束条件，评测方式(Special Judge、交互题)，题目的描述以及评测的数据版本。其通过关联表支持多标签（labels）和多语言（language）。

比赛实体（contest）指的是组织教学、作业、普通考试和正式比赛的集合体。私有密码验证、基本的比赛信息。

判题记录实体（judge）记录每次代码提交的元数据，即源代码、编程语言、耗时、耗空间、判题机 IP 和最终的判题结果（AC、WA、TLE、RE、CE 等）。

测试点详情实体（judge_case）：对单个提交来说，记录它对于每一个具体的测试输入输出样例（Test Case）所表现出的运行情况

比赛提交记录实体（contest_record）是比赛模块的主要数据流水，专门用来记录比赛或者作业期间的提交。动态计算 ACM 罚时就是产生实时排行榜，触发自动重新打分。

权限管理架构为 user_info 和 role 之间存在多对多的关系，而 role 又和 perm

之间也存在多对多的关系。由此就创建起了灵活的安全访问控制。

题库与竞赛联动：problem 与 contest 实体通过 contest_problem 形成多对多关联。一个题目可以被多次引用到不同的竞赛或者作业中，在竞赛内部有一个独立的展示 ID（display_id）。

4.4.3 用户与权限模块数据设计

用户和权限模块属于系统安全体系中的重要部分，主要是对用户的身份信息进行管理、对角色进行权限控制、对访问进行授权。本系统使用基于角色的访问控制模型（Role-Based Access Control，RBAC）来实现权限管理，用用户、角色和权限三者之间的关系来完成系统的功能访问控制。相比于直接给用户分配权限的方法来说，RBAC 模型可以有效地减少权限管理的复杂程度，在多角色的情况下也具有更好的可维护性以及扩展性。

系统会把用户信息、角色信息和权限信息分门别类地存放在不同的数据库表里，用关联表来实现多对多的关系。用户表（user）用来保存账号、密码、昵称、邮箱、状态等基本的身份信息，角色表（role）用来定义系统中各种类型的用户角色，比如普通用户、题目管理员和系统管理员等，权限表（permission）用来描述具体的权限，即题目管理、比赛管理、用户管理、判题查看等操作的权限。由于一个用户可以拥有多个角色，而一个角色又可以关联多个用户，所以系统用用户角色关联表（user_role）来保存用户和角色的多对多关系。角色和权限之间也用角色权限关联

表（role_permission）来映射，从而达到对角色权限的统一管理。

用户登录成功之后，系统会依照用户的角色信息去查询对应的权限集合，然后生成包含用户身份以及权限信息的访问 Token。后续请求到达网关或者业务服务的时候，会根据 Token 中的用户角色和权限信息来完成接口级别的访问控制。为了减少频繁的权限查询给数据库带来的压力，系统把用户的 Token 以及一些权限数据存入到 Redis 中，从而达到快速的身份认证和在线状态管理的目的。

另外为了保证权限数据的一致性、安全性，在用户角色变更、权限修改等操作之后，系统会自动清除对应的缓存数据，保证以后的访问可以得到最新的权限信息。本模块用用户、角色、权限三级关系模型来实现系统的访问控制逻辑解耦，并且给以后新增加角色类型和功能权限留有良好的扩展空间。

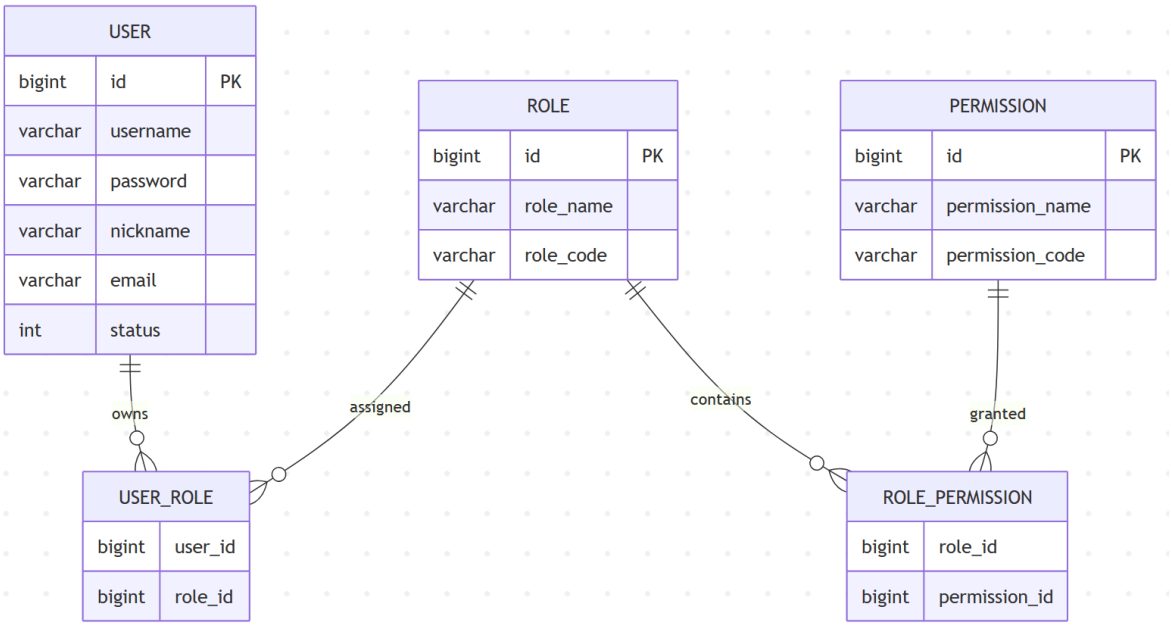


图 4-X 用户与权限模块实体关系图

4.4.4 题目与比赛模块数据设计

题目与比赛模块属于在线评测系统的业务模块，主要是对题目进行管理、组织比赛、关联提交记录并统计排行榜的功能。题目模块主要是保存题目描述、标签分类、测试用例配置、评测限制这些基本的信息，比赛模块则是组织多道题目举行的竞赛活动，对用户的提交记录、比赛排名和比赛状态进行统一管理。由于在线评测系统需要同时支持公开练习和正式比赛这两种运行模式，所以在数据库的设计上使用的是题目独立存储、比赛引用引用的方式来实现题目资源的复用以及比赛数据的隔离。

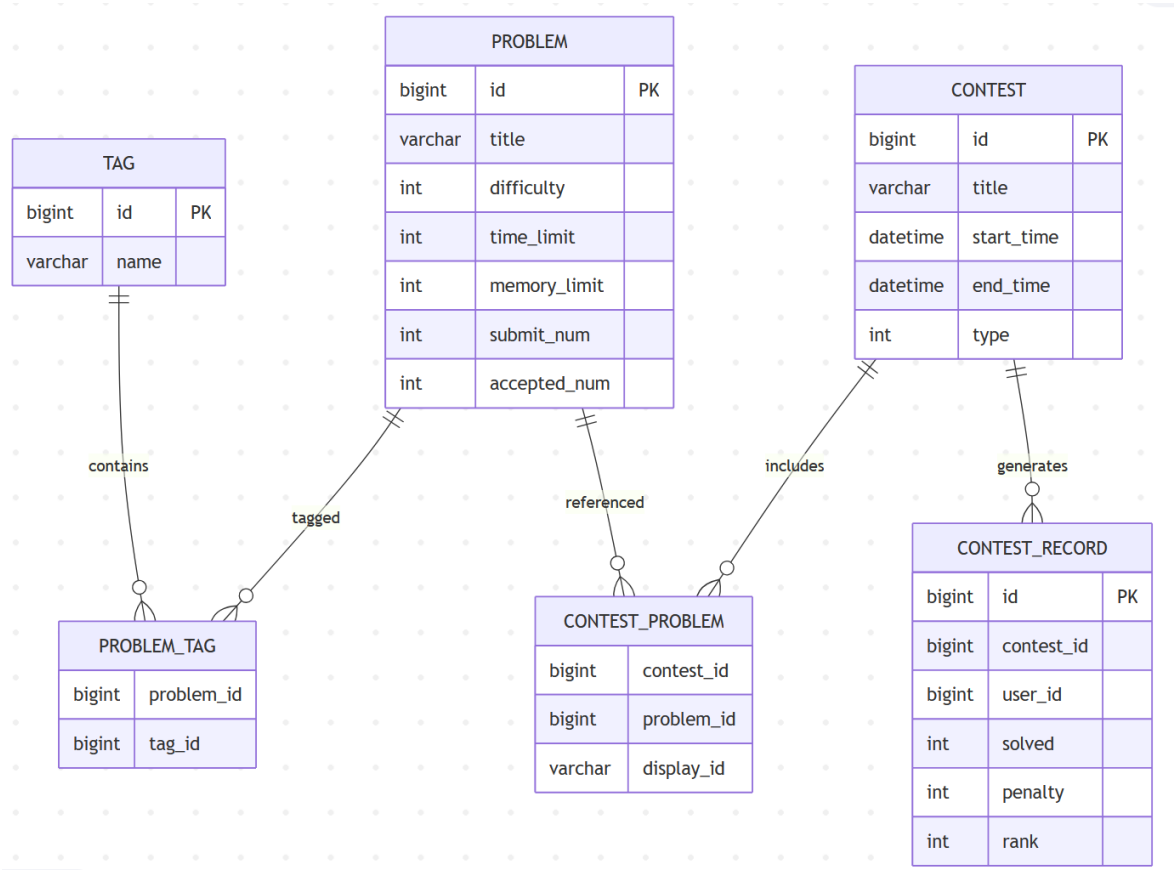


图 4-X 题目与比赛模块实体关系图

题目数据结构设计上，题目表（problem）用来保存题目标题、时间限制、内存限制、题目难度、提交次数、通过次数等基本资料，并且记载题目所处的测试用例版本以及判题方式这些评测设置。由于一个题目可以有多个标签，一个标签也可以关联多道题目，所以系统用题目标签关联表（problem_tag）来建立题目和标签之间的多对多关系，从而支持题目分类检索和标签筛选的功能。标签表(tag)用来保存算法分类、题目类型、知识点等信息，方便之后的题目推荐和统计分析。

比赛模块的数据库表为比赛表（contest），用以存放比赛标题、比赛开始时间、比赛结束时间、比赛类型和访问权限等信息。因为一场球赛会有多道题目，同一个题目也会被不同的比赛使用，所以系统用比赛题目关联表（contest_problem）来保存比赛和题目之间的多对多关系。为了对用户在比赛中提交的情况以及排名进行记录，系统创建了比赛记录表（contest_record），用来保存用户在比赛期间提交的次数、通过的题目数、罚时、最后的排名等信息，为排行榜的产生和比赛结果统计提供数据支持。

数据访问优化上，因为比赛列表、题目详情、排行榜等是高频访问数据，所以系统用 Redis 来对相关的统计信息和热点数据做缓存，从而减小数据库的压力，加快页面的响应速度。对于需要短时间内快速刷新的场景，系统用短时缓存和定时刷新相结合的方式来提高查询速度，同时又尽量减少由于数据不一致而产生的不良影

响。通过对题干和比赛模块进行分析可知，系统中用到的是多对多的关系模型来实现题目资源共享、比赛活动组织分离，为以后系统的各种竞赛形式以及大量的在线评测做好了数据结构上的准备。

4.4.5 用户与权限模块数据设计

判题模块属于在线评测系统的主要部分，主要作用是对用户的提交信息加以记载，对测试点的执行结果进行管理，并且要维持好分布式判题节点的状态。因为判题过程中需要对频繁的状态进行更新，对多个测试点的结果进行统计，同时还需要在分布式判题机之间协同完成评测工作，所以系统在数据结构的设计上主要考虑评测流程的状态管理、测试点级别结果的存储以及高并发环境下数据的访问速度，从而保证系统可以稳定地完成大量的在线评测任务。

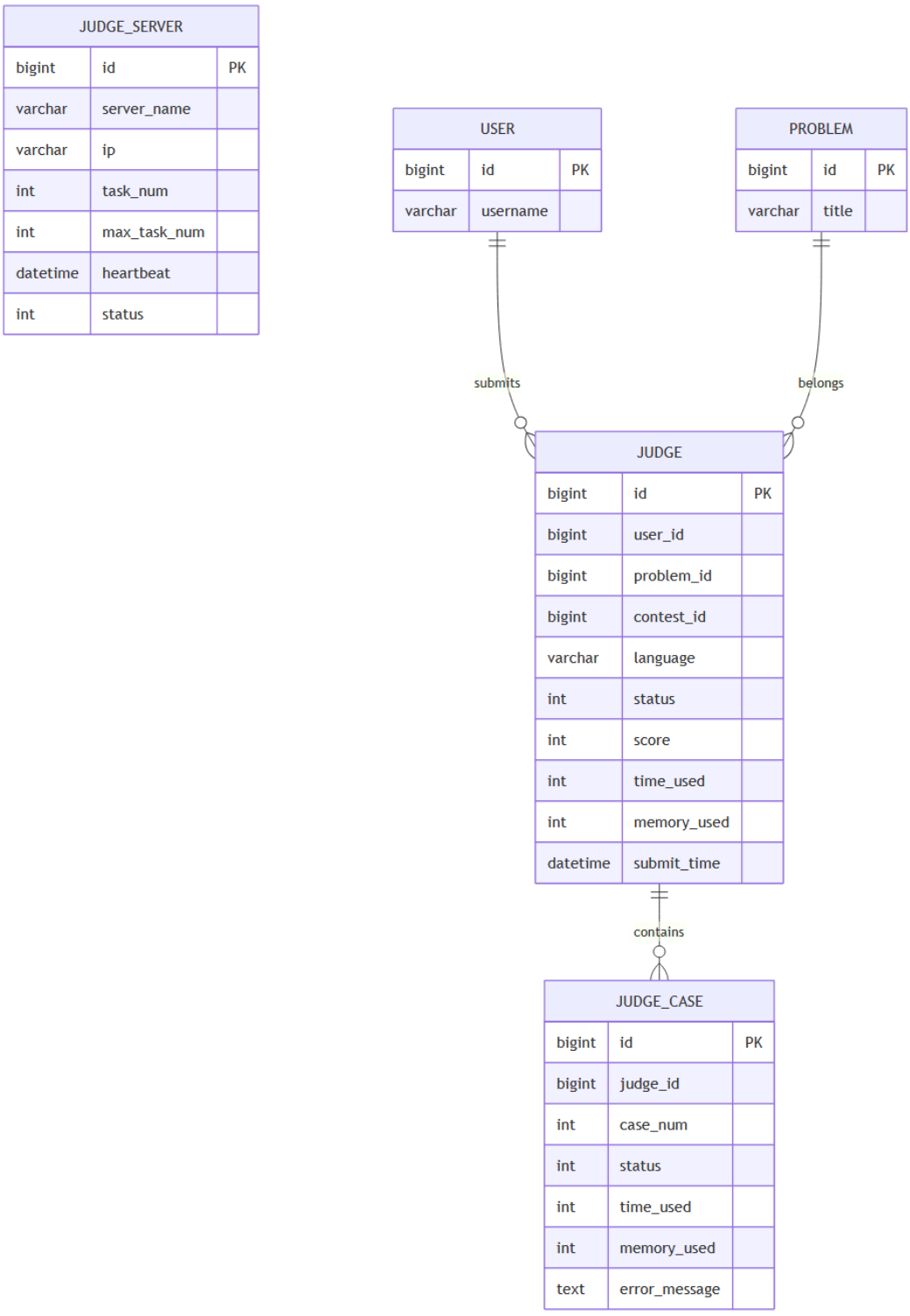
系统用判题表（judge）来记录用户的提交信息，即提交的代码、使用的编程语言、所属题目、提交的时间和最终的判题结果等。判题表同时维护评测状态字段，用于描述提交在不同阶段的生命周期状态，例如等待评测（PENDING）、编译中（COMPILING）、评测中（JUDGING）、评测完成（FINISHED）以及系统错误（SYSTEM_ERROR）等。通过显式状态流转的设计来达到对评阅任务的异步调度、并发控制的目的，并且可以很方便的在前端实时获取评测进度。

由于一道题目一般会有多处测试点，不同的测试点会产生不同的运行结果，所以系统用 judge_case 表来记录每一个测试点的独立执行信息，即测试点编号、运行

时间、内存占用、判题结果和错误输出等数据。判题服务在所有的测试点执行完毕之后，会对测试点的结果进行汇总，然后计算出最终的得分、最大耗时和最终状态写回到判题表中。与只保存最后结果的方法不同的是，测试点级别数据结构具有对错误进行详细分析的能力、对 OI 模式部分得分的统计以及评测结果的可视化展示等功能。

分布式判题架构上，系统创建了一个判题节点表（judge_server），用以保存各个判题机节点的运行状况及资源信息，包含节点名称、IP 地址、当前任务数目、最大任务容量以及心跳时间这些数据。业务服务用消息队列把判题任务发送到 RabbitMQ 上，各个 Judge Server 节点异步消费任务来进行评测。使用判题节点信息表可以实时查看判题机的运行情况，给之后的任务调度、节点负载均衡以及异常节点剔除打下基础。

另外，在高频的判题场景中为了降低数据库的负担，系统把测试用例信息、题目评测配置和一些判题的状态缓存在 Redis 里，用缓存双重检测机制和分布式锁来避免缓存被击穿的情况。判题结果更新采用状态先更新、结果后回写的策略来保证评测过程的一致性，防止由于任务意外终止造成数据状态的错误。本模块利用提交记录、测试点结果、判题节点这三个数据模型创建了一个完备的在线评测流程支持系统，给系统的高并发、多语言、多题型判题提供了一个稳定的架构。


JUDGE_SERVER
bigint id PK
varchar server_name
varchar ip
int task_num
int max_task_num
datetime heartbeat
int status
USER
bigint id PK
varchar username
PROBLEM
bigint id PK
varchar title
submits
belongs
JUDGE
bigint id PK
bigint user_id
bigint problem_id
bigint contest_id
varchar language
int status
int score
int time_used
int memory_used
datetime submit_time
contains
JUDGE_CASE
bigint id PK
bigint judge_id
int case_num
int status
int time_used
int memory_used
text error_message

判题模块实体关系图

5 系统核心功能的实现

5.1 用户认证与授权功能实现

用户认证属于系统安全体系的重要部分，它的主要任务就是对用户的身份进行校验，在认证通过之后创建起一个统一的登录状态。由于本系统同时支持平台账号登录和旧版 HUSTOJ 会话登录两种认证方式，所以系统使用 Spring Security 构建了可以扩展的认证链路，用自定义过滤器（Filter）、认证提供器（AuthenticationProvider）、认证成功/失败处理器来实现统一认证流程。

5.1.1 认证请求处理流程

当用户提交登录请求之后，请求就会先经过 IpLoginRateLimitFilter 登录限流过滤器来限制单位时间内异常登录的次数，防止暴力破解攻击。随后，请求进入 LoginAuthenticationFilter，系统根据请求头中的 loginType 字段识别当前登录方式，并执行对应认证逻辑。

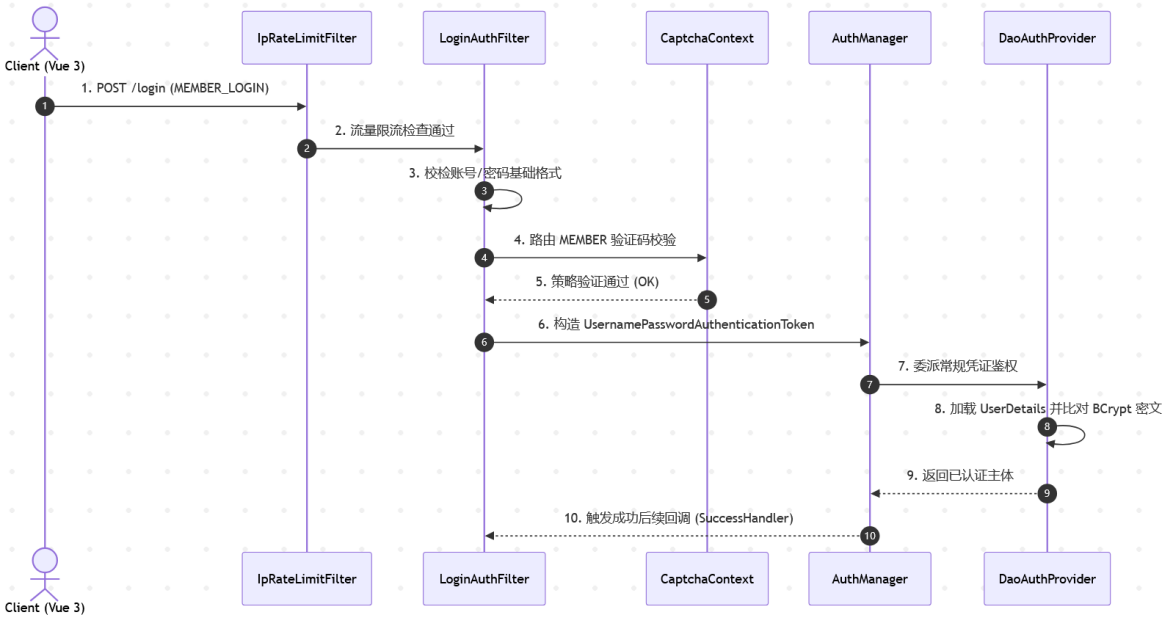
当登录类型为普通账号密码登录（MEMBER_LOGIN）时，系统先对用户名和密码参数的合法性进行校验，再调用 CaptchaContext 完成验证码校验。验证码验证成功之后，系统构造 UsernamePasswordAuthenticationToken 并提交给 AuthenticationManager。认证管理器再调用 DaoAuthenticationProvider 来完成用户的认证工作，其中通过 UserDetailsService 来查询用户信息，同时使用

PasswordEncoder 来对密码进行校验。

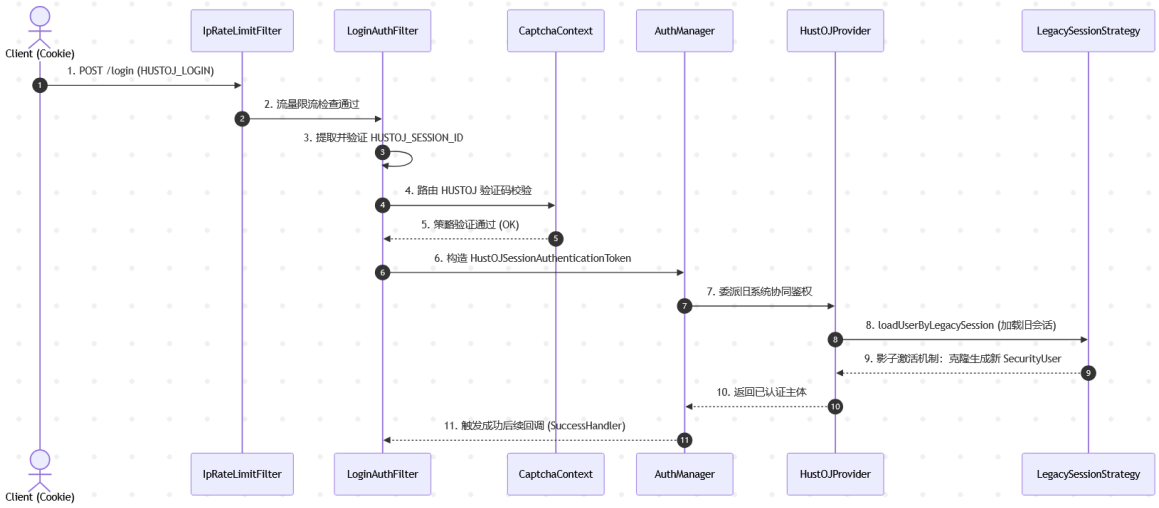
当登录类型为旧版 HUSTOJ 会话登录(HUSTOJ_LOGIN)的时候，系统先读取 HUSTOJ_SESSION_IDCookie，然后验证 SessionId 合法性。接着又进行验证码的校验，再构造出 HustOJSessionAuthenticationToken 提交给 AuthenticationManager。认证管理器调用 HustOJSessionAuthenticationProvider 对旧系统会话进行认证，认证提供器使用 LegacySessionStrategy 加载旧版系统中用户的登录状态信息，从而实现旧系统登录状态的兼容。

如果登录类型不合法，参数校验失败或者认证过程中出现异常，系统就统一进入到 AuthenticationFailureHandler 进行失败处理，然后给客户端返回错误信息。

认证请求处理流程如图 5-X 所示。



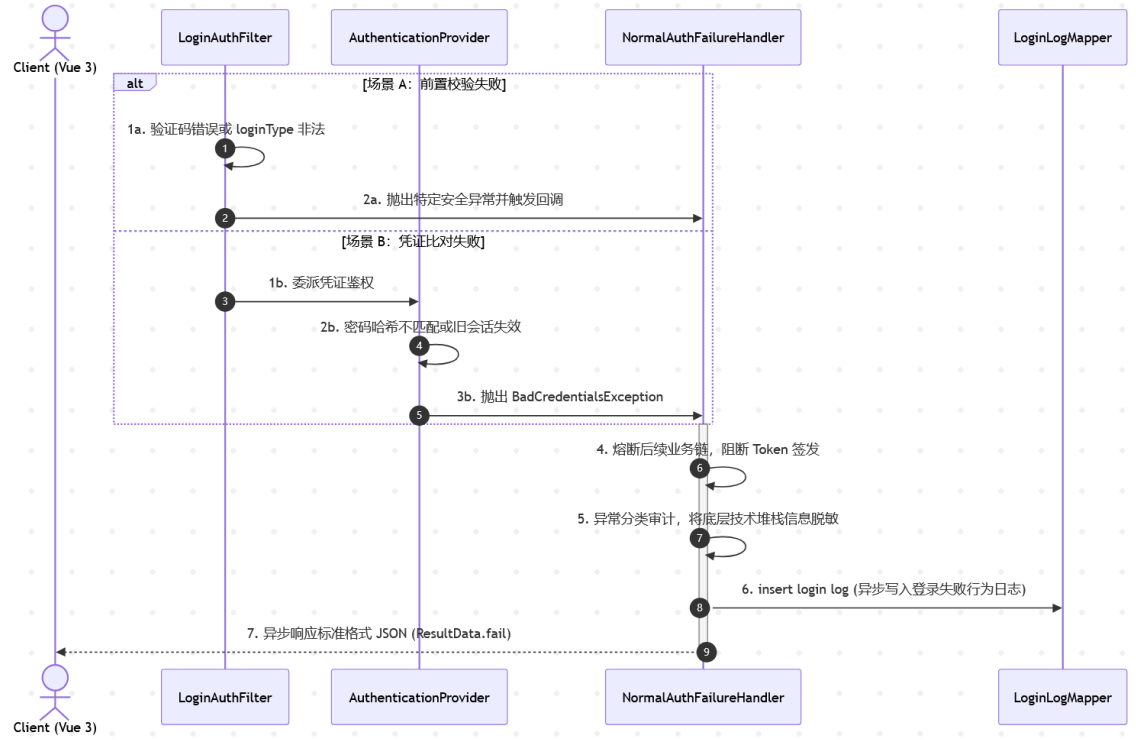
常规账号密码认证流实现



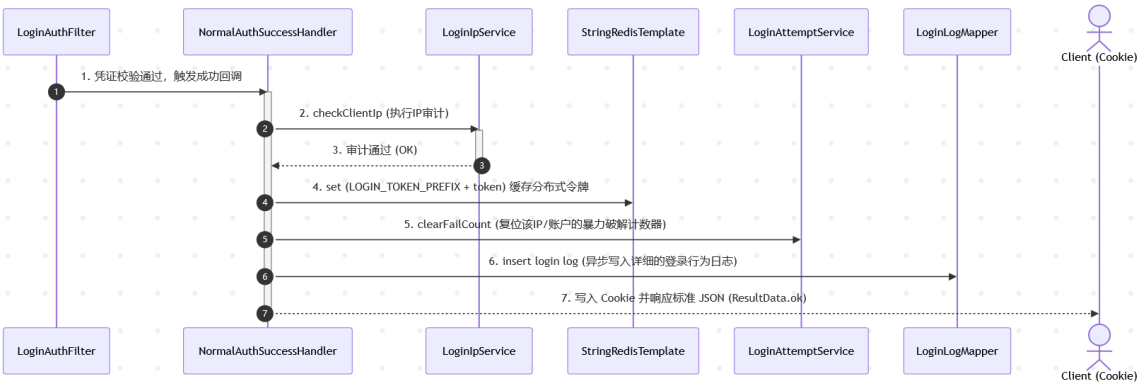
旧系统 HUSTOJ 协同认证流实现（HUSTOJ_LOGIN）

5.1.2 登录成功和失败处理流程

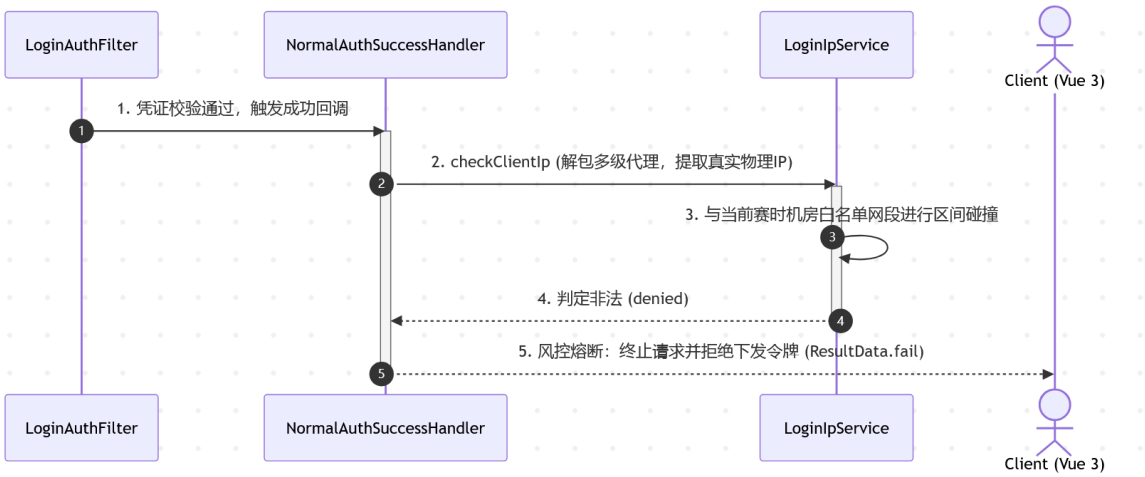
登录失败处理阶段（FailureHandler 阶段）当认证过程中任何一个节点出现问题，例如验证码不符、密码哈希和系统内存储的密码不符、session 过期、非法 loginType 请求头等的时候，就会立即产生相应的系统安全异常。该异常将被 NormalAuthenticationFailureHandler 强制捕获，熔断后续流程，并将错误信息高度脱敏后封装为标准的 ResultData.fail() 报文以异步 JSON 密流回传，确保分布式多节点的接口响应一致性。



登陆成功处理（SuccessHandler 阶段）当以上场景一或者场景二成功输出已经认证的 SecurityUser 主体之后，系统就触发了 NormalAuthenticationSuccessHandler 闭环控制。系统先委派给 LoginIpService 剥离多层代理得到客户端真实的物理 IP，再和当前赛时绑定的校内机房网段做区间碰撞。如果确认是代考或者异地越权，就立即熔断请求，拒绝签发令牌。利用 StringRedisTemplate 缓存分布式 Token，调用 LoginAttemptService 重置历史失败计数，由 LoginLogMapper 异步写入 MySQL 登录审计日志，最后向前端下发 ResultData.ok()报文。



IP 风控拦截实现



令牌签发与行为状态审计实现

5.1.3 验证码校验流程实现

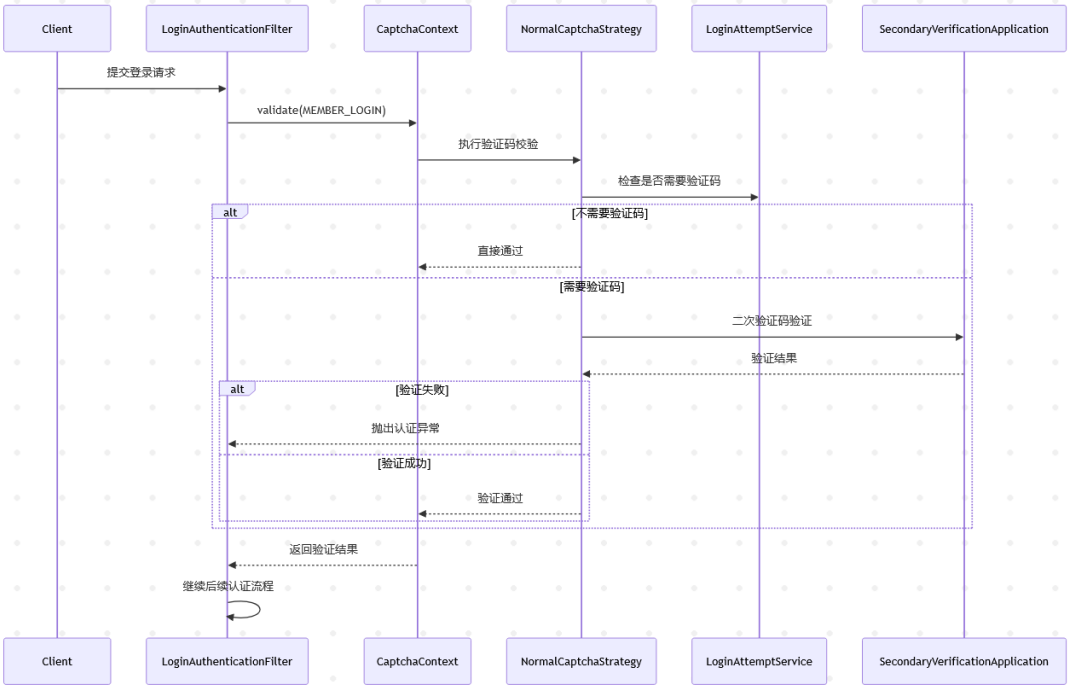
为了减少暴力破解、批量恶意登录请求给系统安全造成的不良影响，在登录认证的过程中加入了动态验证码校验的方法。不同于传统的所有的请求都必须强制使用验证码的方案，本系统根据登录失败次数来决定是否需要进行验证码的验证，在保证安全性的基础上尽可能地减少正常用户登录过程中不必要的交互开销。验证码模块全部使用策略模式来完成，用 CaptchaContext 统一调度不同的验证码校验策

略，来适应平台账号登录和HUSTOJ会话登录这两种认证场景。

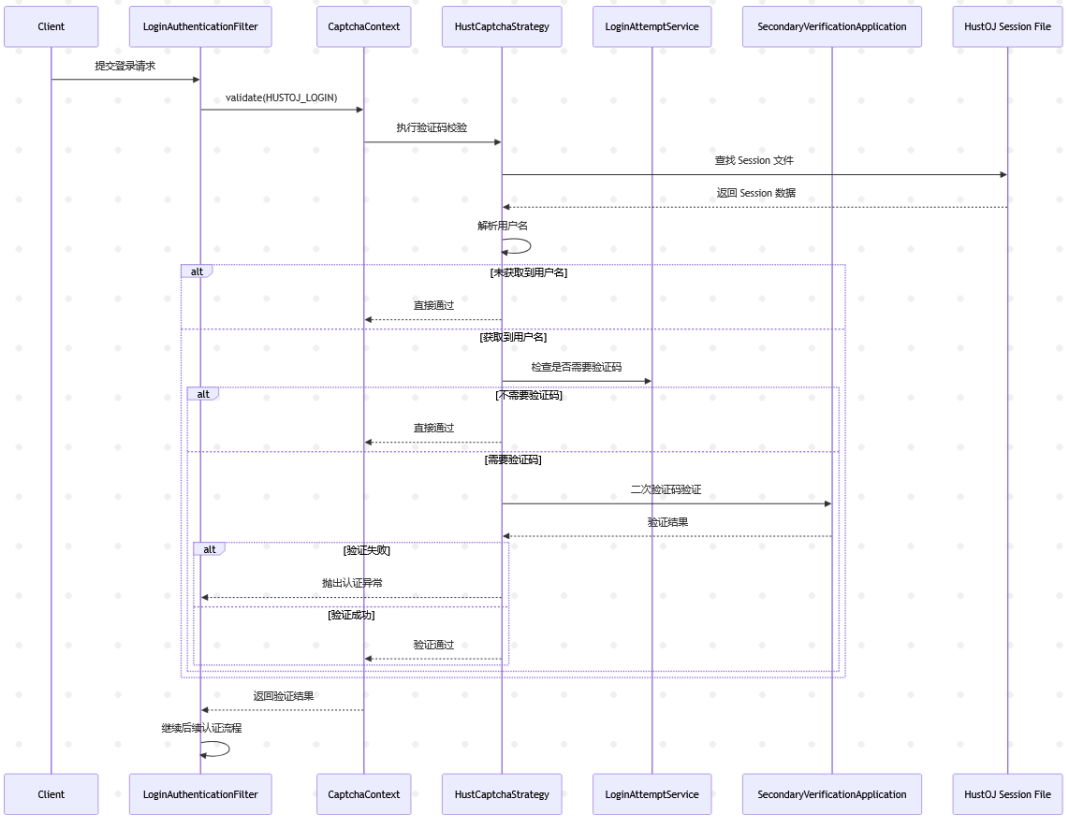
普通账号密码登录（MEMBER_LOGIN）场景中，LoginAuthenticationFilter解析登录请求之后，就会调用CaptchaContext执行验证码校验逻辑。验证码上下文根据登录类型来选择NormalCaptchaStrategy进行具体的验证。系统先从LoginAttemptService查询当前用户是否达到验证码触发阈值；如果用户登录失败次数没有超过限制，就直接跳过验证码验证流程，继续后面的认证操作。当失败次数超过阈值的时候，系统会要求用户提交验证码Token，然后调用SecondaryVerificationApplication进行二次验证。当验证码验证失败的时候，系统抛出InsufficientAuthenticationException并结束认证流程。

HUSTOJ会话登录时，由于用户还没有输入用户名，所以系统要先从SessionId中获取旧版系统中用户的登录信息。HustCaptchaStrategy 会读取 HUSTOJ Session文件，并解析对应用户名；随后同样调用 LoginAttemptService 判断是否需要验证码验证。当达到验证码触发条件的时候，会调用二次验证服务来校验验证码Token，验证通过之后才开始执行后面的认证流程。

与传统的固定验证码方案相比，本系统使用的是以失败次数为触发条件的验证码动态安全策略，从而在保证系统的安全性的同时，也简化了正常的用户登录操作。验证码模块采用策略模式来把不同的登录方式验证逻辑分离开来，为之后添加短信验证码、邮箱验证码等认证方式打下了良好的基础。



普通登录验证码校验流程图



HUSTOJ 会话验证码校验流程图普通登录验证码校验流程图

5.2 分布式判题功能实现

分布式判题系统是在线评测平台的核心模块，接收用户提交的评测任务，在判题节点执行代码，收集测试点执行结果，最后把结果回写到数据库中。由于系统各个微服务彼此独立，每一个服务直接访问自己的数据库，没有通过服务之间的调用来获取数据，所以缓存未命中时，各个微服务都会直接去访问自己的数据库来获取需要的信息。这样就实现了微服务的自治性，并且依靠缓存（Redis）来降低数据库的访问压力。

5.2.1 提交任务与异步调度

分布式判题流程的开始就是用户提交评测任务。系统通过前端界面接收用户提交的代码以及提交信息，然后将这些信息发送到 gateway-server 上。网关服务负责路由请求到 judge-service，在这个过程中会验证用户的 Token 是否合法，只有合法的用户才可以提交评测任务。

系统在判题服务内部先去缓存 Redis 中存储的用户信息、题目信息、比赛信息等高频数据，从而减轻数据库的压力。如果缓存没有命中，就直接去对应的微服务数据库里拉取需要的数据。接着，在数据库中创建一个新的 Judge 记录（状态为 PENDING），用来表示当前提交的任务生命周期状态。比赛场景下系统也会产生 ContestRecord 来记录用户比赛的提交情况以及成绩统计。

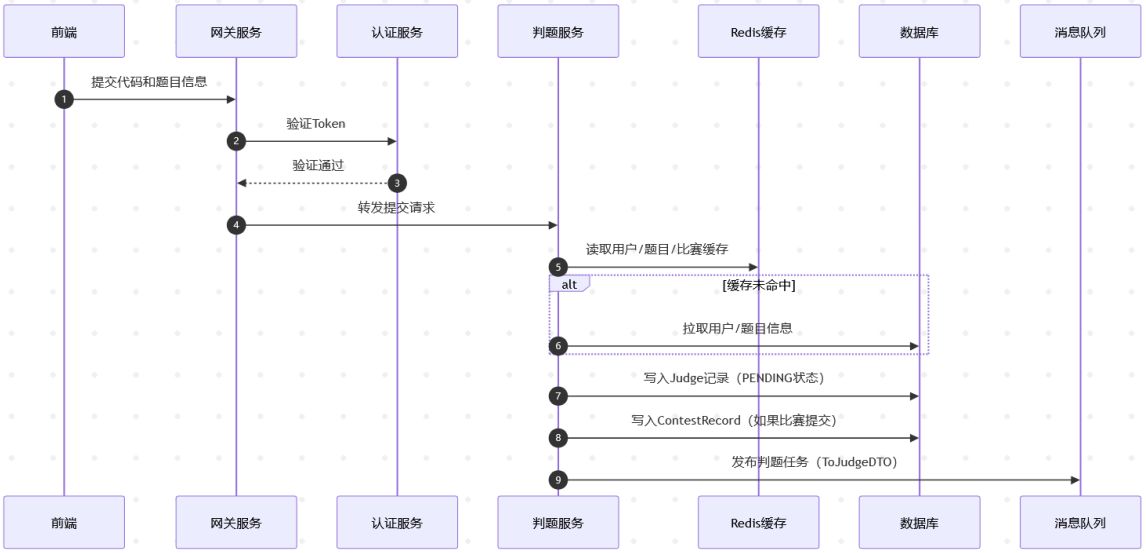


图 5-X 提交任务与异步调度流程图

完成基础数据写入后，judge-service 将判题任务封装为 ToJudgeDTO 消息，并通过 RabbitMQ 发布到判题任务队列（JudgeTaskExchange），实现异步化任务调度。因此前端提交操作和判题执行被分离了，用户不用等代码执行完就可以得到提交确认。异步队列机制能很好地平滑高峰期的判题压力，对大规模竞赛以及高并发提交环境都有很好的稳定效果。

5.2.2 判题节点执行

判题节点（JudgeServer）是分布式判题系统中实际执行提交的评测任务的执行单元。每一个判题节点都是独立运行的，通过消息队列异步地从 judge-service 接收到 ToJudgeDTO 的任务，然后按照任务的内容来获取题目、测试用例和用户提交的代码信息。

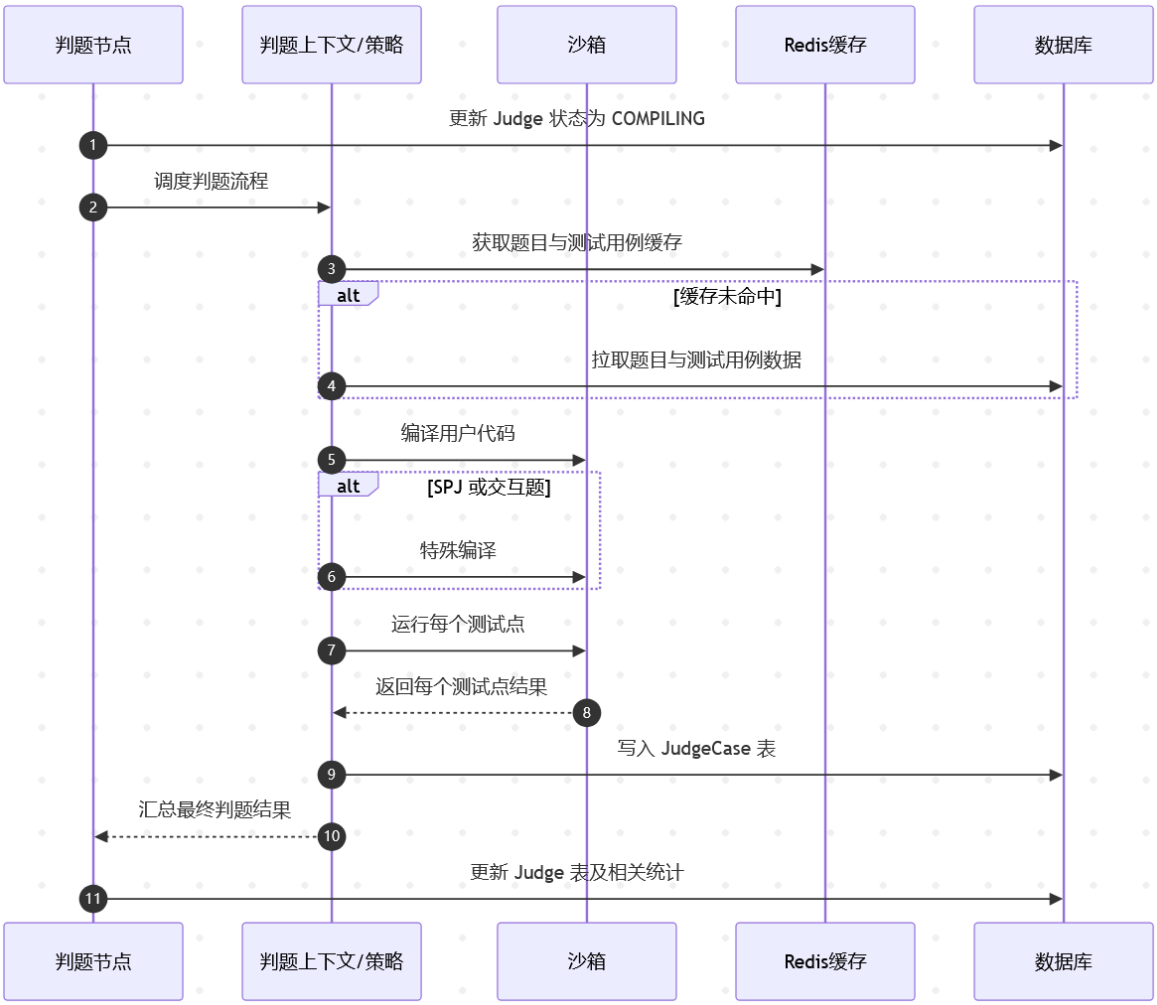


图 5-X 判题节点执行流程图

任务到达判题节点之后，节点会先做 Token 验证工作，保证任务的来源是合法的，防止恶意注入或者伪造请求。然后系统把 Judge 记录的状态改成 COMPILING，记录当前的判题节点信息，标记任务执行者。接下来，节点通过 JudgeContext 与 JudgeStrategy 调度判题流程，包括：

题目信息和测试用例的获取，从 Redis 缓存中获取题目评测信息以及测试用例，如果缓存没有命中就去数据库或者文件系统中获取数据。

用户代码编译，调用 Sandbox 执行用户代码的编译，对特判题（SPJ）或者交互题（Interactive）做特殊的编译。

测试点执行时，判题节点一个接一个地执行每一个测试点，每个测试点的运行结果包含状态、时间、内存占用和错误信息，存入 JudgeCase 表中。

结果汇总与回写，汇总所有的测试点得分、总耗时、总内存占用，更新 Judge 表及相关统计表，即题目统计表、用户已解题信息表、比赛记录表等，保证排行榜以及统计数据与提交结果一致。

整个流程保证了判题节点是独立的、可以并行的，可以支持大规模并发提交，用 Redis 缓存来降低对数据库的访问压力。每一个判题节点都是独立完成任务的，微服务之间不需要直接通信，从而构建出一个高可用、可扩展的系统。

5.2.3 测试用例与题目数据获取

分布式判题系统里测试用例和题目信息都是评测任务的主要数据。由于系统使用微服务自治的方式进行设计，各个服务都是独立地访问数据库的，所以判题节点在执行任务的时候必须保证可以获取到完整的题目信息以及对应的测试用例，这样才能对题目进行准确的评测。

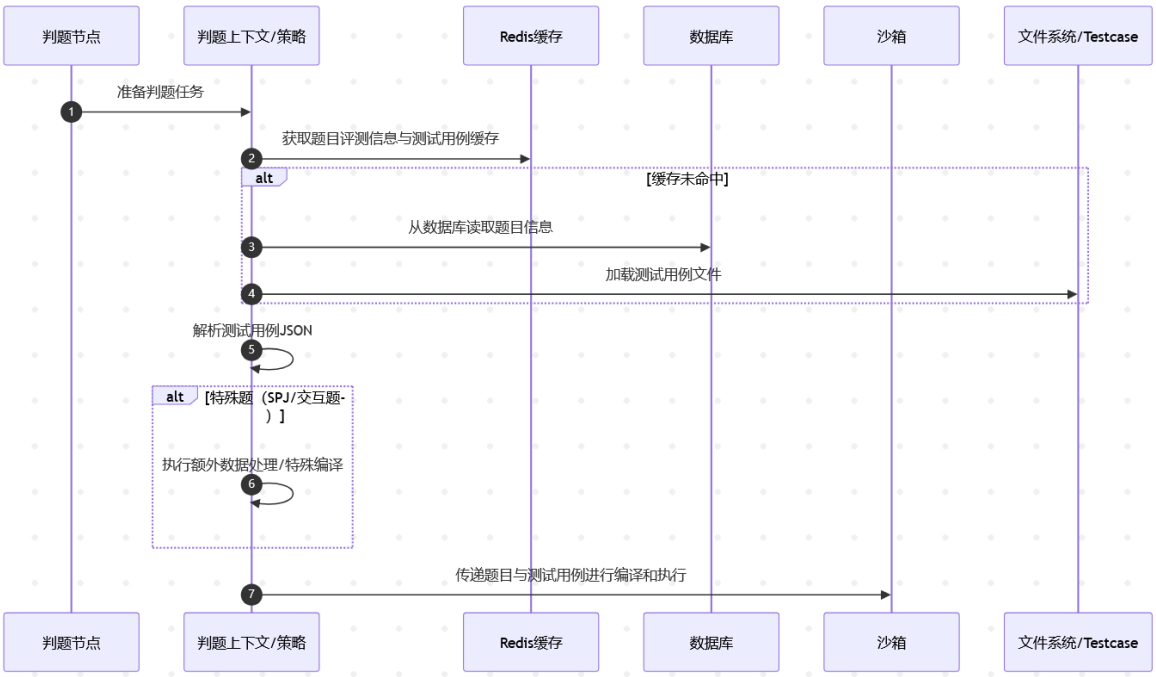


图 5-X 测试用例与题目数据获取流程图

首先判题节点会去 Redis 缓存里查找题目评测信息和测试用例，缓存里有题目配置、时间/内存限制、语言要求以及每一个测试点的输入输出数据。缓存未命中时，节点直接去数据库或者文件系统取数据，保证微服务的独立性，不需要跨服务调用就可以得到数据。

获取数据之后，判题节点把测试用例 JSON 解析成内部评测结构，即每一个测试点的输入、标准输出、运行约束、特殊配置（SPJ 或者交互题）。对 SPJ 或者交互题来说，在判题之前系统会对数据做额外的处理或者特殊的编译，由 JudgeStrategy 根据题目类型来选择相应的逻辑。解析之后题目、测试用例信息就会传给 Sandbox 进行编译和执行。

5.2.4 安全沙箱执行（Sandbox）

分布式判题系统中，用户提交的代码要在一个隔离的安全环境里执行，不能对判题节点或者操作系统的稳定造成任何影响。系统使用的是开源 Go-Judge 项目提供的沙箱服务，每一个判题任务都在独立的执行环境中运行，用户代码不能直接访问系统的资源，限制 CPU、内存以及系统调用权限来达到安全隔离的目的。

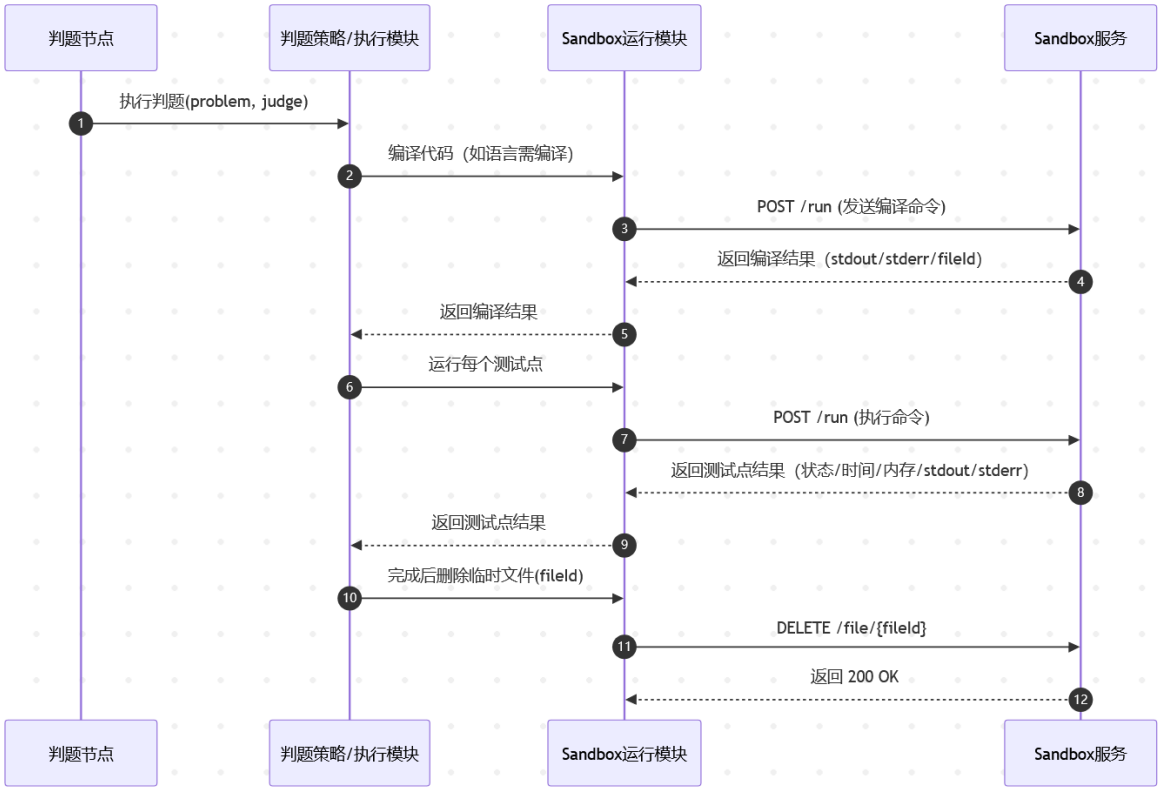


图 5-X 安全沙箱执行流程图

判题节点（JudgeServer）收到任务后，将任务信息传递给判题策略与执行模块（JudgeStrategy / JudgeRun）。本模块根据语言类型来决定是否要编译，然后调用沙箱执行编译。编译命令经由 Sandbox Service 的/run 接口发出，得到的是编译

的结果，即标准输出、错误信息以及生成的可执行文件标识。编译完成之后，判题策略模块就开始对每一个测试点进行执行，依次将执行请求发送到 Sandbox Service 上，然后从 Sandbox Service 上获取每个测试点的运行结果，即状态、执行时间、内存占用和标准输出/错误信息。所有的测试点执行完毕之后，系统就会把生成的临时文件删除，用 Sandbox Service 的文件删除接口释放资源。

整个沙箱的运行过程实现了用户程序的安全隔离，可以统一处理各种语言、各种题型（包括特判题 SPJ、交互题）评测任务，在分布式环境下可以对大量的评测任务进行快速执行。

5.2.5 特殊判题和交互判题处理

在线评测系统中为了实现 ACM 模式下特判题（SPJ）和交互题（Interactive），在判题策略模块中加入了相应的特殊判题方法。本模块用 judge 策略(JudgeStrategy、JudgeRun)来统一管理用户的程序和特殊的程序编译、运行。

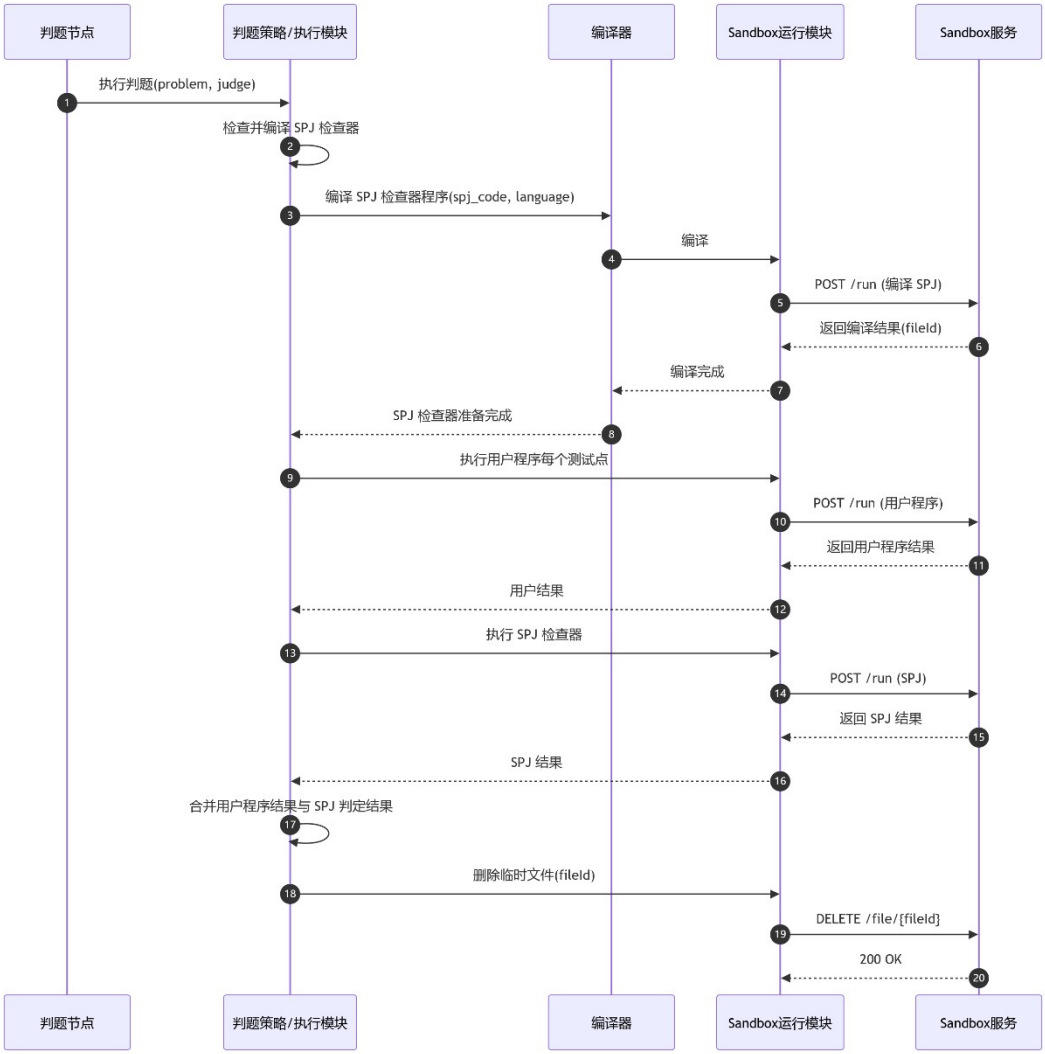


图 1：SPJ 特判题执行流程

判题节点收到任务后，首先调用 checkOrCompileExtraProgram() 检查是否需要为当前题目编译额外的判题程序（如 SPJ checker 或交互器）。题目类型为SPJ时，系统会先编译SPJ检查器程序，使用Sandbox服务执行编译命令，然后返回可执行文件标识。然后判题策略模块就依次对用户的程序进行每一个测试点的运行，把运行结果同SPJ检查器程序的判定结果结合起来，得到最终的测试点评测结果。

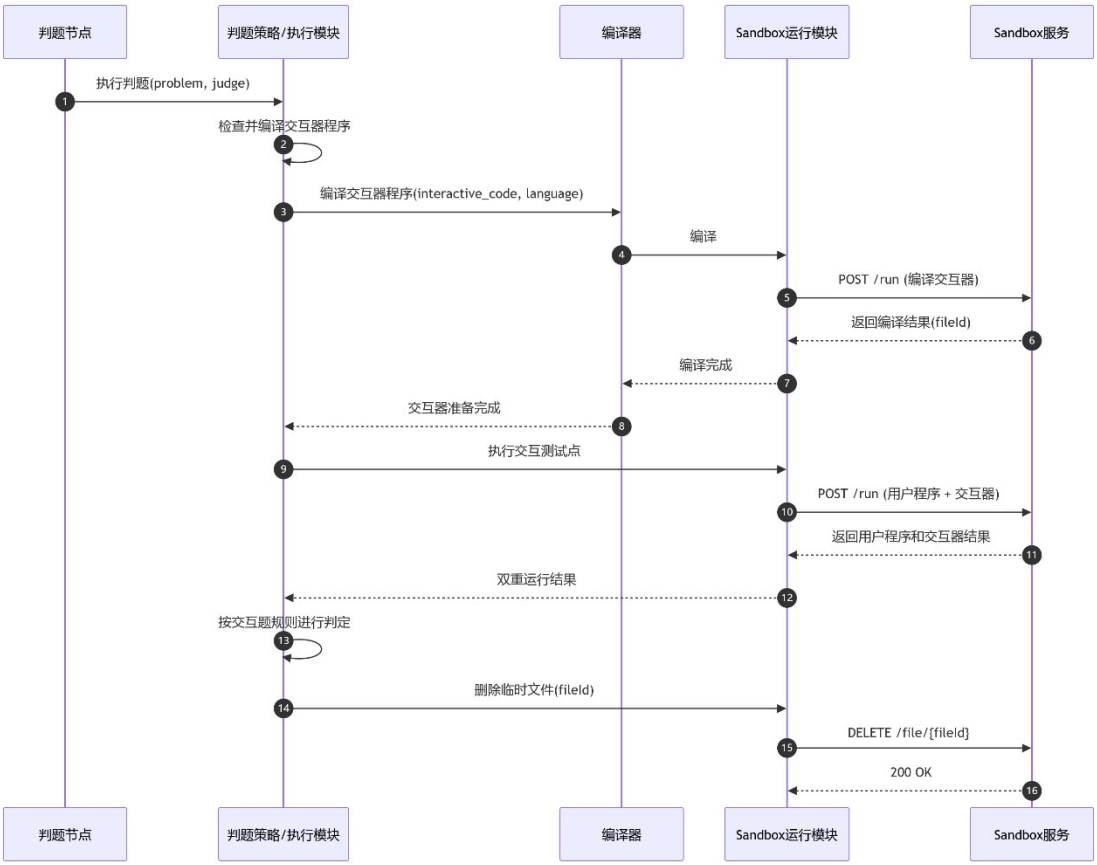


图 2：交互题执行流程

交互题系统先编译交互器程序，然后用 Sandbox 执行生成可执行文件。判题策略模块在每一个测试点执行的时候，会同时运行用户程序和交互器，获取用户和交互器的两个运行结果，然后根据交互题规则来评判，得到最终的状态。执行完毕之后，系统会把编译出来的临时文件全部删除，从而释放出系统所占用的资源。

5.2.6 判题结果回写与统计更新

分布式判题系统中，用户程序执行完毕之后，系统就要对所有的测试点结果加以汇总，然后把最终的结果回写到数据库当中去，还要更新有关的统计信息，从而

保持提交状态、排行榜以及题目统计数据的一致性。系统采用微服务自治的设计思想，每一个判题节点都是独立的，回写操作是在数据库事务控制下进行的，不需要显式的分布式锁。

具体实现上写回操作依靠的是数据库事务、重试机制以及幂等写入来保证并发的安全性。回写入口在 JudgeContext.updateOtherTable(...)中，内部调用 updateUserSolvedProblemTable(...)和 updateContestSolvedProblem(...)，这两个方法都带有@Transactional 注解，还使用了@Retryable 来处理并发事务冲突。写入数据库的时候用到的语句是“INSERT…”ON DUPLICATE KEY UPDATE″的方式（insertOrUpdate），利用数据库唯一约束实现幂等更新，避免重复写入造成的数据不一致。

另外系统中使用了 Redisson 分布式锁来实现缓存读写，即题目评测信息的缓存。判题结果回写不依靠 Redisson 锁，用事务和幂等 SQL 来保证安全性、一致性。

回写流程中的判题策略模块会把每一个测试点的结果加起来得到总分、最大耗时以及内存使用情况。判题节点最后把最终的结果写入 Judge 表，同时更新题目统计数据、用户已经解题的记录以及比赛记录（ContestRecord）。所有的操作都在事务控制之下进行，以此来保证高并发环境下数据的一致性以及系统的稳定性。



图 5-X 判题结果回写与统计更新流程图

5.3 比赛与排行榜功能的优化实现

在线评测系统中的比赛、排行榜等都是核心的业务模块，在线评测系统中这些模块的好坏直接决定着用户体验的好坏，在高并发的比赛场景中也不例外。系统在设计和实现的时候使用了很多优化的方法，通过缓存、分布式锁、双重检测等方式保证排行榜数据的实时性和准确性。

5.3.1 研究背景与问题分析

比赛、排行榜功能是在线评测系统中最基本的功能，主要是对比赛进度的显示、成绩的竞赛、用户的反馈意见等起到重要的作用。与普通查询不同的是，排行榜要综合处理题目配置、参赛用户信息、提交记录、判题结果和竞赛规则（首杀、罚时等），所以它的业务逻辑比较复杂，计算成本也比较高。同时比赛进行的时候也会产生明显的流量热点，大量的用户频繁刷新排行榜，在比赛的后期访问量会达到高

峰，容易造成请求集中、系统抖动、响应变慢甚至超时等现象。

从系统角度看，排行榜查询存在以下典型挑战：

(1) 高并发与热点读请求：排行榜属于读密集场景，用户刷新频繁，短时间内请求量极大，若直接依赖数据库查询，将导致数据库连接耗尽、查询排队与响应延迟上升。

(2) 排行榜构建复杂度高：排序需要处理多维度数据，包括题目映射、用户提交记录聚合、首杀判定、罚时计算、题目状态维护等，单次计算成本显著高于普通业务查询。

(3) 实时性与一致性矛盾：判题结果持续写入，排行榜数据具有动态变化特性。若追求强一致，每次写入都触发即时更新，将引发写放大和锁竞争；若不更新缓存，则会产生短暂不一致，影响实时性。

(4) 缓存击穿与雪崩风险：排行榜属于高热度数据，若缓存过期或失效，可能造成大量并发请求同时访问数据库，形成缓存击穿甚至雪崩。

以上问题造成排行榜模块成为系统性能瓶颈的主要原因，因此优化策略要在这三个方面之间取得平衡。因此本章就排行榜功能的缓存策略、并发控制以及一致性权衡进行优化的设计和实现。

5.3.2 总体优化思路

为了使比赛和排行榜的功能达到高并发、高计算成本以及一致性要求的目的，

本文提出了一个以缓存优先、锁防击穿、短时最终一致为核心思想的总体优化方案，在系统实现的过程中又坚持了读路径减压、写路径去耦、构建计算可控的原则。整体策略是从请求路径、缓存策略、并发控制、一致性模型这四个方面来展开的。

（1）读路径优化：缓存优先、最小化数据库访问

排行榜查询属于高频读操作，所以用 Redis 作为第一层访问层。请求进入时先读取缓存，缓存命中就立即返回结果，不进行后面的计算和数据库访问。大大减轻了DB的负担，使系统高峰时期依然可以有较好的响应速度。

（2）并发控制：分布式锁防止缓存击穿

排行榜属于热点数据，如果缓存过期，就会出现大量的并发请求同时回源数据库的情况。为了避免缓存击穿，用 Redisson 的分布式锁来控制同一个比赛的排行榜构建的互斥操作。只有获取锁的线程来重建缓存，其他的线程等待或者返回错误，从而防止数据库雪崩。

（3）短时最终一致：平衡实时性与性能

写入频繁的时候会产生强一致性导致的写放大、同步阻塞的问题。因此排行榜缓存使用短的TTL（5秒），虽然会有轻微的延迟，但是可以提高吞吐量和稳定性。即使出现短暂的不一致，在很短的时间内就会自动修正，符合比赛排行榜业务对于实时性可以接受的范围。

（4）双重检查机制：避免重复构建与锁内浪费

在获取锁之后再进行缓存检查，防止其他线程已经完成了构建但是当前线程还在继续计算，从而避免重复计算的开销。该机制可以很好地解决高并发的时候锁内计算的浪费问题。

（5）数据构建策略：缓存数据为主、计算逻辑集中

排行榜的建立要依靠题目、用户、提交记录这三个数据。优先使用已有的缓存，必须从数据库中回源的时候才进行；计算逻辑放在服务层，不进行复杂的聚合操作以减少DB的负担，提高可维护性。

5.3.3 关键实现设计

客户端调用排行榜接口之后，服务端不会直接去数据库里查或者做复杂的计算，而是会先使用固定Key（contestRank:{id}）去访问Redis。如果命中，说明榜单在短时间内已经建立起来并且仍然有效，服务端只更新 serverTime 作为响应时间的基准，就可以返回结果。这条路径没有任何数据聚合、排序的开销，在高并发环境下可以达到微秒级别的响应速度，是本优化方案的主要性能提升点。

当缓存没有命中时，系统就进入了防击穿的模式。因为排行榜是热点数据，当高并发的时候直接回源数据库会造成瞬时压力激增，所以使用Redisson分布式锁来保证同一个比赛的榜单重建只由一个线程执行。成功获得锁之后再进行二次缓存读取（double-check），防止其他线程在当前线程排队的时候已经完成了回填，造成重复构建。如果二次读取没有命中，才进入构建阶段。如果获取锁失败，就立刻返

回错误或者降级响应，防止并发请求同时回源数据库造成雪崩。此阶段关键作用在于约束并发重建数量与保护数据库。

形成阶段为计算最为繁杂的阶段。服务端先获取比赛题目、参赛用户、提交记录三类数据，优先从缓存服务中获取，需要的时候再从数据库中回源。获取提交记录之后，先剔除掉题目已经被删除的记录，保证排行榜语义正确；接着统计每道题的首杀记录，为后面的用户标记提供依据；再以用户为维度聚合题目状态，得到“是否解题、提交次数、解题时间、首杀标记”等细粒度信息；最后计算总解题数和罚时得到 RankVO 结构。建立之后，结果存入 Redis，并且设置了短的 TTL（比如 5 秒）来保证实时性可以接受。本阶段就是缓存优先、计算集中、短 TTL 回写的一个闭环。

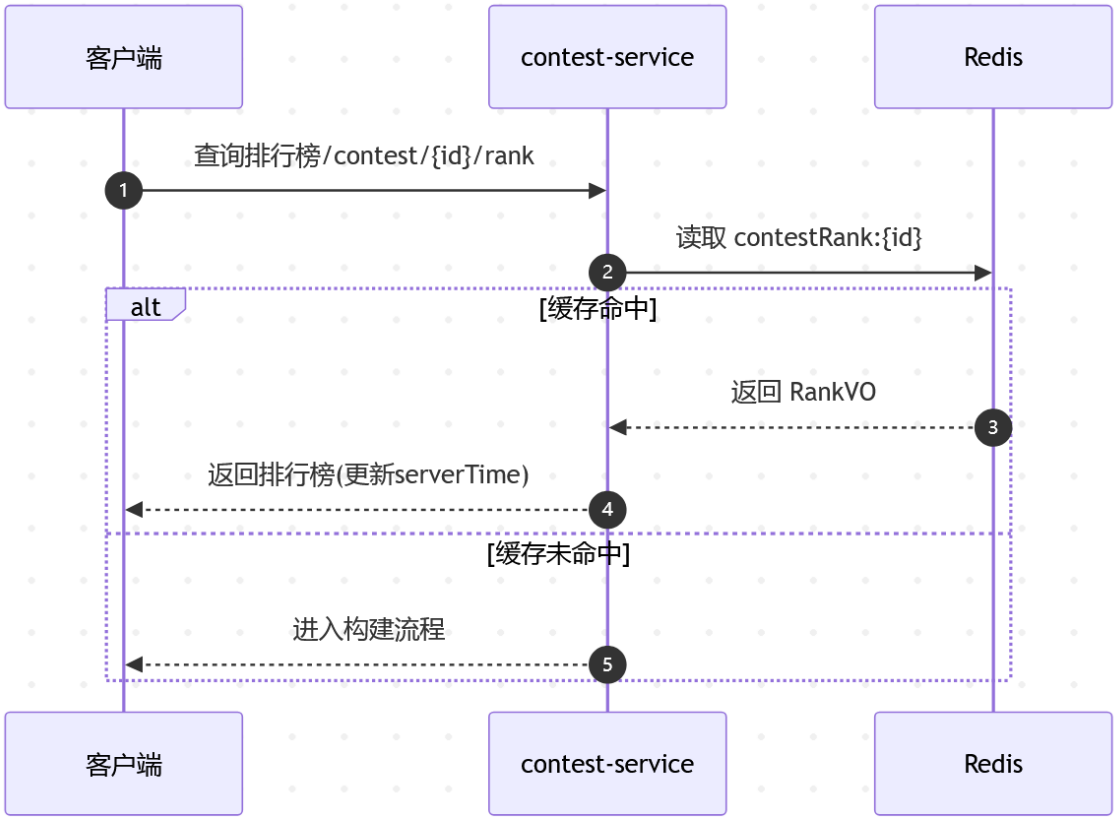
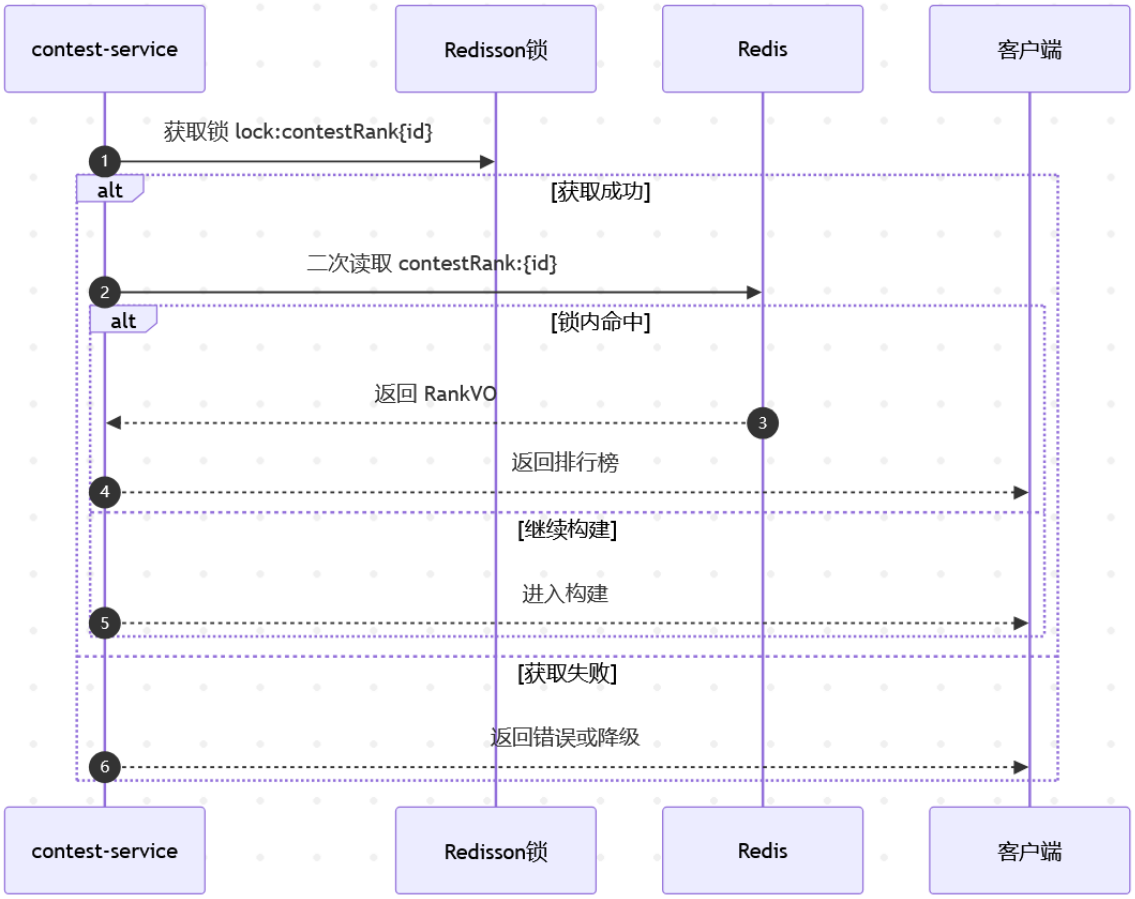


图 1：入口与缓存命中路径


contest-service
Redisson锁
Redis
客户端
获取锁 lock:contestRank{id}
1
alt
[获取成功]
二次读取 contestRank:{id}
2
alt
[锁内命中]
返回 RankVO
3
返回排行榜
4
[继续构建]
进入构建
5
[获取失败]
返回错误或降级
6
contest-service
Redisson锁
Redis
客户端

锁保护与缓存重建入口

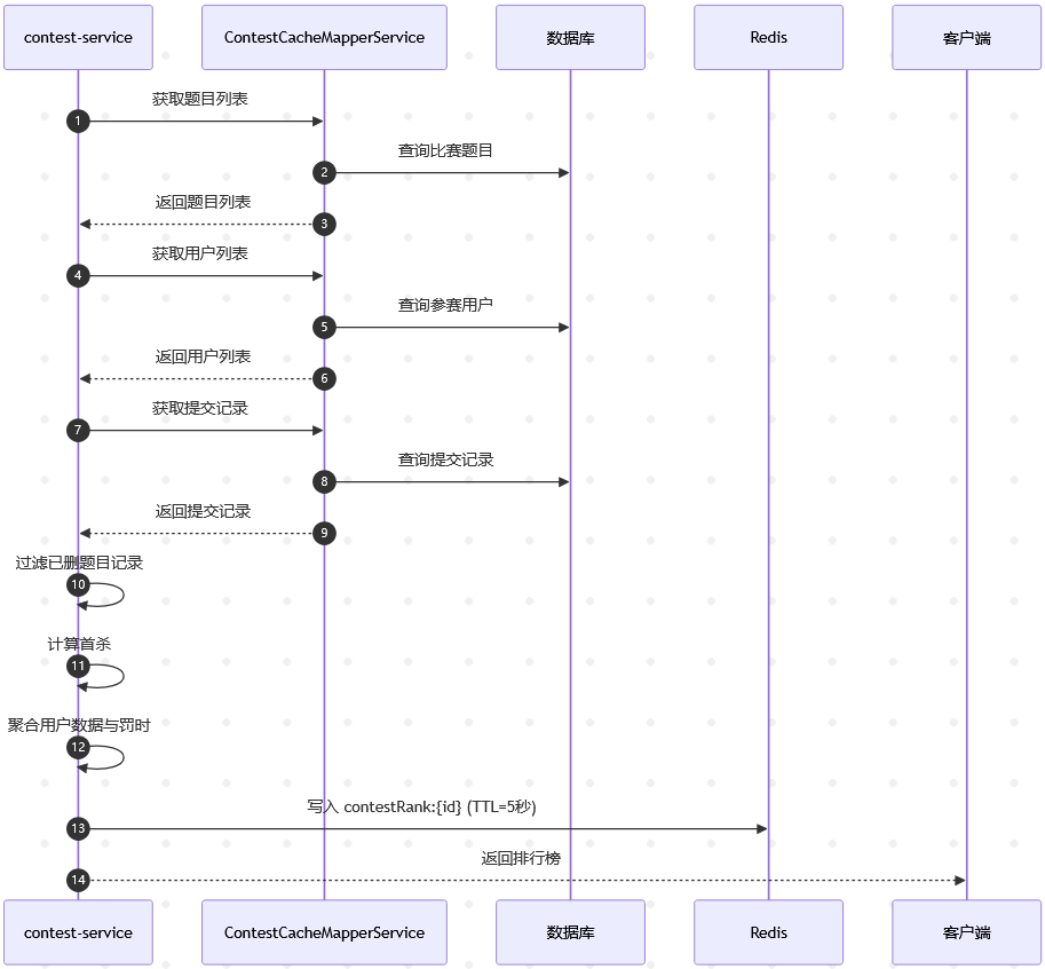


图 3：排行榜构建与缓存写回

7 系统测试与实战分析

7.1 测试环境与测试方案

项目	配置
CPU	Intel Xeon E5620 2.4GHz x2
内存	64GB
OS	Ubuntu 22.04
JDK	Java 21
数据库	MySQL 8
Redis	Redis 7.4.0
RabbitMQ	RabbitMQ 4.3.1
Sandbox	go-judge 1.12.0
Docker	Docker 23.0.3

本系统采用 Linux 环境部署，微服务通过 Docker Compose 进行管理。Redis 用于缓存与排行榜存储，RabbitMQ 用于分布式判题任务调度，Go-Judge 负责用户程序隔离执行。

测试方案设计：功能测试、压力测试、比赛实战测试。

7.2 系统功能测试

7.2.1 用户认证功能测试

用户认证模块是对登录、Token 校验、验证码验证、权限控制等进行测试的。系统支持普通账号密码登录与兼容旧版 HUSTOJ Session 的双登录模式，并通过 Spring Security 实现统一认证流程。

测试时对正确的账号密码、错误的密码、验证码错误和 Token 失效等进行测试。结果说明系统可以正确地识别出用户的身份，在认证失败的时候会返回相应的错误信息。频繁登录失败的用户要进行验证码二次验证来减少暴力破解带来的风险。

权限控制测试中，不同的角色用户访问受限接口的时候，系统可以对权限进行正确的校验，未授权的请求会被 Gateway 或者 Spring Security 拦截，保证系统的接口安全。

常规账号密码认证（MEMBER_LOGIN），测试项主要是验证本地新注册用户标准认证链路。测试人员在前台登录界面输入系统中已经存在的标准学号、经过高强度加密的正确密码和实时生成的图形验证码。系统后台 CaptchaContext 策略中心首先对验证码进行碰撞，判定无误后激活 DaoAuthenticationProvider。该组件调用本地 UserDetailsService 检索用户实体，并利用 PasswordEncoder 执行 BCrypt 哈希密文比对。测试结果表明，系统可以签发分布式 Token，并且用异步 JSON 密流写入到前端 Cookie 中，测试结果为 PASS。

7.2.2 评测功能测试

测试判题功能的目标是检验系统识别各种判题结果的能力，例如 Accepted、Wrong Answer、Compile Error、Time Limit Exceeded、Memory Limit Exceeded 和 Runtime Error 等。

在测试时，系统也提交了一些有代表性的测试程序。通过死循环程序来检验超

时限制，用大量的内存申请程序来检验内存限制，用非法数组访问程序来检验运行时错误处理。测试结果说明 Sandbox 可以正确地对程序的运行资源进行限制，并且可以返回相应的判题状态。

此外，系统还对特殊判题（SPJ）和交互题（Interactive）进行了测试。SPJ 测试中系统可以正确调用 SPJ 检查器对用户的输出进行判断；交互题测试中系统可以正确运行用户程序和交互器，完成交互过程中数据的传输以及结果的判定。

7.2.3 比赛与排行榜功能测试

比赛模块主要是测试比赛创建、提交限制、排行榜刷新、首血计算、封榜等功能。

在测试的过程中，多用户同时提交代码，系统可以及时更新排行榜，对用户的题目通过次数和罚时信息也进行了正确的统计。第一个通过题目的人可以被系统正确地标记为First Blood。比赛进入封榜阶段之后，新提交的结果不会马上出现在排行榜上，在滚榜阶段统一更新。

为了检验排行榜高并发环境下系统的稳定性，系统对排行榜接口做了频繁访问的测试。通过实验可以得出结论，Redis 缓存、Redisson 分布式锁可以有效地减小数据库的压力，排行榜接口在高并发的情况下仍然可以保证较快的响应速度。

下图展示了比赛排行榜页面：



排名	用户名	姓名	总时间	AC	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	ujnacm202559	材2515程明达	1727	15	※1st 0 1 try	※1st 159 1 try	※1st 167 1 try	106 1 try	215 2 tries	38 1 try	※1st 103 1 try		6 tries	※1st 65 1 try	50 1 try	※1st 59 5 tries	※1st 48 2 tries	※1st 19 1 try	※1st 226 10 tries	※1st 11 2 tries	※1st 121 2 tries
2	ujnacm2025164	集成2502王皓	928	8	2 1 try			74 1 try	42 4 tries	16 1 try					55 3 tries		223 1 try	105 4 tries		211 3 tries	
3	ujnacm202533	图灵2501卢恩雨	260	6	3 1 try			86 1 try	21 1 try	13 1 try					67 1 try		3 tries	50 2 tries	2 tries		
4	ujnacm2025224	数科2501孔德	458	6	2 1 try			105 1 try	40 2 tries	15 1 try					60 1 try		136 5 tries	12 tries			3 tries
5	ujnacm202555	数科2501李鸿阳	673	6	3			※1st 34	54	37				215	70		3 tries				

7.3 分布式判题压力测试

7.3.1 高并发提交测试

为了检验系统在高并发情况下稳定性的表现，在测试时用压力测试工具来模拟大量的用户同时提交代码。测试的主要目的是验证提交接口的响应时间、RabbitMQ 队列积压情况、JudgeServer 节点的任务处理能力。

测试结果表明，当并发量比较低的时候，系统可以很快地完成任务的调度和判题；当并发量逐渐增大时，RabbitMQ 队列会暂时堆积任务，但是各个 JudgeServer 节点可以同时消费队列中已经有的判题任务，系统整体仍然可以保持稳定的运行。

由于判题使用异步处理机制，用户提交代码之后不需要等待判题完成就可以立即得到提交结果，所以在高并发环境下系统仍然可以保持较好的用户体验。

7.3.2 RabbitMQ 异步调度分析

系统用 RabbitMQ 做为判题任务的异步解耦。JudgeService 收到提交请求之后，把判题任务封装成消息发到判题队列里，各个 JudgeServer 节点并行消费。

测试结果说明 RabbitMQ 可以很好地处理突发的大量数据提交。当比赛进入到提交高峰的时候，队列里就会出现短时间的消息堆积，但是随着 JudgeServer 不断消费，队列很快就能恢复到正常状态。

异步调度的方式使提交接口和判题执行之间没有太大的耦合，从而提高系统的整体吞吐量以及系统的横向扩展性。

7.3.3 Sandbox 隔离执行测试

Sandbox 模块主要是对用户程序在隔离环境中执行的安全性进行测试。测试时系统提交了死循环程序、大内存申请程序和非法系统调用程序来检验 Sandbox 对于 CPU、内存、系统调用的限制能力。

测试结果说明 Sandbox 可以正确地结束超时的程序，可以控制程序所占的内存。当用户程序访问非法的资源或者执行危险的操作的时候，Sandbox 就会立刻中止程序的运行，以此来保证判题节点的安全。

另外，在高并发环境下，不同的 Sandbox 实例可以同时运行不同的用户程序，任务之间不会互相影响，证明了 Sandbox 隔离机制是有效的。

7.4 缓存与排行榜优化效果分析

7.4.1 Redis 缓存优化效果分析

系统在比赛排行榜、题目统计、用户信息等很多地方使用了 Redis 缓存来减少数据库的访问压力。

测试结果表明，在没有使用缓存的时候，排行榜接口每次请求都需要进行多表关联查询，响应时间长；使用 Redis 缓存之后，大部分请求可以直接从缓存中获取数据，接口响应时间明显降低。

系统使用短的 TTL 来保证排行榜的实时性，并且用 Redisson 分布式锁以及双重检查的方式来防止高并发时出现的缓存击穿情况。

下图展示了缓存优化前后的排行榜接口响应时间变化：



7.4.2 分布式锁与缓存一致性分析

当排行榜缓存没有命中时，就用 Redisson 分布式锁来控制缓存重建的过程。获取锁成功后的线程负责从数据库加载数据并生成排行榜，其它线程则等待缓存生成完成之后再读取缓存。

系统使用了双倍检查（double check）来保证数据的完整性，即拿到锁之后再去

查缓存里有没有别的线程更新了它。

这样就使得系统可以有效地降低数据库重复查询，并且还能保证缓存和数据库数据的一致性。

7.5 实战比赛运行分析

7.5.1 比赛运行情况

本系统已经被用于程序设计竞赛中，可以顺利地完成济南大学第八届程序设计新生赛。比赛期间系统要同时处理大量的用户提交和排行榜访问请求，对于系统的稳定性和判题性能都有较高的要求。

比赛中各个 JudgeServer 节点不断地从 RabbitMQ 中消费判题任务，Redis 缓存可以快速响应排行榜请求，整个系统运行稳定，没有出现严重任务堆积或者服务不可用的情况。

7.5.2 系统运行效果分析

分布式判题结构可以很好地减小单个节点的负担，进而提高整个系统所能够处理的信息流。RabbitMQ 异步调度机制对于比赛高峰期大量的提交请求有着较好的处理效果，而 Redis 缓存也可以大大减少排行榜接口访问数据库的压力。

Sandbox 隔离执行机制可以保证恶意程序或者异常程序不会对判题节点的稳定造成影响，从而保证比赛期间系统安全可靠地运行。

7.6 本章小结

本章对系统做了功能测试、压力测试和实战运行分析。由上表可见，本系统可以完成用户认证、代码评测、比赛管理、排行榜更新等主要功能，在高并发情况下也表现出较好的稳定性和性能。采用基于 RabbitMQ 的异步调度、多 JudgeServer 分布式判题结构、Redis 缓存机制等方法，使系统具有更好的处理能力、更快的响应速度。从系统实际运行的结果来看，在真实环境下它的可靠性是可得的。

8 总结与展望

8.1 总结

本文主要对在线评测系统（Online Judge，OJ）的设计与实现做了详细的论述。从系统架构上来说，使用微服务的设计思想，把用户认证、判题、题目管理、比赛管理等各个功能模块分别做成独立的服务，从而达到服务自治以及易于扩展的目的。使用 REST 接口、消息队列来实现微服务间的异步通信，从而使得系统具有高可用性、松耦合的特性。

系统创建了一个分布式判题架构，用 RabbitMQ 来做异步任务调度，多节点的 JudgeServer 一起完成判题任务。通过使用 Go-Judge 的安全沙箱来保证用户代码是在隔离环境当中运行的，并且支持多种语言以及不同类型的题目（普通题、SPJ、交互题）的评测。高并发提交的时候，系统可以保证判题的稳定以及任务的吞吐量。

系统还使用了 Redis 缓存优化排行榜、题目统计以及用户信息的读取，在短的 TTL 以及 Redisson 分布式锁的基础上加上双重检查机制来达到缓存命中率高、数据库一致性的目的。经过系统的功能验证和实战测试之后，系统表现稳定，可以正确处理所有的判题状态，在线更新排行榜，可以应用于实际的比赛当中，比赛高峰期依然能保持良好的性能与稳定性。

因此本系统功能完备，性能优良，高并发处理能力，可以满足高校程序设计教学以及比赛的要求。

8.2 展望

虽然系统功能、性能都取得较好效果，但是还有进一步改进的空间。未来可以在多语言、多环境的支持上加以扩展，增加对于新兴编程语言的支持，并且可以提供多沙箱执行环境来提高隔离性。系统用增量判题、动态调度、智能缓存优化等方法来降低高并发时的延迟和数据库负担，提高总的吞吐量。

除此之外，前端用户体验以及可视化效果还存在着提高的空间，可以通过实时排行榜动画、提交历史可视化、成绩分析图表等形式来提高用户的用户体验。从系统安全的角度出发，可以加强对恶意代码、资源占用、分布式攻击等的防护措施，并且可以加入异常检测、自动报警等功能。

系统在教育、竞赛的应用中还可以继续扩展，在教育领域可以实现课程作业自动判题、在线实验、题库管理等，在竞赛领域可以采用分布式计算的方式，实现多校联合竞赛的判题工作，提高多校联合竞赛的判题效率和稳定性。经过以上改进之后，系统就会成为一种高效的、安全的、可扩展的综合在线评测平台，在实际应用中发挥更大的作用。

结　论

本文对在线评测系统（OnlineJudge，OJ）的设计与实现做了系统的、实践的工作。采用微服务架构拆分之后，系统就完成了用户认证、分布式判题、比赛管理以及排行榜这些主要的功能模块，从而达到了服务自治和容易扩展的目的。判题模块使用 RabbitMQ 异步调度、多节点并行处理的方式，用 GoJudge 沙箱来实现用户代码的安全隔离执行，可以满足用户的各种语言、各种题型的评测需求，在高并发的情况下也能够保证稳定性的表现。

系统中使用了 Redis 缓存策略以及 Redisson 分布式锁，并且使用了双重检查机制来保证缓存命中率以及数据库的一致性，从而大大降低了数据库的压力，提高了排行榜以及统计功能的实时性。经过功能测试、压力测试以及实际比赛运行检验，系统可以正常地、有效地对大量的提交任务进行处理，在线评测、竞赛中也能正常使用。

因此本系统满足了功能完整性的要求、性能稳定的要求、高并发处理的要求这三个方面的要求，为高校程序设计的教学、比赛提供了技术上的可行方案和实际案例。同时系统具有可扩展性以及优化的空间，以后可以加入多语言的支持、智能判题策略、前端可视化、安全防护等，从而提高系统的能力和用户体验，构建出一个高效、安全、可扩展的综合在线评测平台。

参 考 文 献

[1] 周立. Spring Cloud 与 Docker 微服务架构实战[M]. 2 版. 北京: 电子工业出版社, 2019: 12-25.

[2] 孙玄. 微服务设计原理与架构[M]. 北京: 机械工业出版社, 2020: 50-75.

[3] Alibaba. Nacos Documentation: A Dynamic Service Discovery, Configuration and Service Management Platform[EB/OL]. 2023 [2026-05-22]. https://nacos.io.

[4] 朱忠华. RabbitMQ 实战指南[M]. 北京: 电子工业出版社, 2017: 30-45.

[5] 付磊, 张益军. Redis 开发与运维[M]. 北京: 机械工业出版社, 2017: 60-85.

[6] PostgreSQL Global Development Group. PostgreSQL 15 Documentation: Structured Query Language (SQL)[EB/OL]. 2022 [2026-05-22]. https://www.postgresql.org/docs/15/sql.html.

[7] Criyle. go-judge: Full-featured sandbox service execution server[CP/OL]. 2023 [2026-05-22]. https://github.com/Criyle/go-judge.

[8] 丁瑞, 侯进, 孙宇博. 基于 Docker 容器技术的在线评测系统设计与实现[J]. 计算机应用与软件, 2020, 37(1): 24-29.

[9] 李振华. 在线评测系统（OJ）的安全沙箱技术研究[J]. 信息技术与网络安全, 2021, 40(3): 15-22.

[10] 尤雨溪. Vue.js 设计与实现[M]. 北京: 人民邮电出版社, 2022: 100-120.

[11] Wathan A. [M]. 2018: 45-70. Refactoring UI.

[12] PrimeTek. PrimeVue Component Suite for Vue.js[EB/OL]. 2024 [2026-05-22]. https://www.primefaces.org/primevue/.

[13] 韦超英.MySQL 数据库备份及恢复方法解析[J].网络安全技术与应用,2019(10):73-74.

[14] 米向荣.高可用 MySQL 同步集群的研究与设计[J].忻州师范学院学报,2019,35(05):26-32.

[15] 刘琼滔.基于 MySQL 数据库的运维工作量综合管理平台的设计与实现[J].电信技术,2019(S1):45-48.

[16] 胡强.MySQL 数据库常见问题分析与研究[J].电脑编程技巧与维护,2019(12):91-92.

致　　谢

历时数月的毕业设计就要结束了，从架构设计到实战演练的过程使我对它有更深刻的认识。在此向在项目开发、赛事保障过程中给予我信任、启迪、反馈的老师和同学们致以最真诚的谢意。

首先衷心感谢我的导师蔺永政老师。他最初给我提供的 C 语言评测系统，是我在探究评测逻辑时开始的。通过本系统的学习，掌握了计算机的基础知识，为以后独立重构系统打下了良好的基础。

特别感谢蒋彦老师，他是学校 OJ 系统的管理员，他不但让我进入到了在线评测系统中，而且给了我很大的自主开发空间。他支持我在这个原有的 HustOJ 体系的基础上，为了解决实际业务问题而开发出一套全新的微服务 OJ 系统。蒋老师提出的建议使得系统由原来的单纯的代码运行平台变成一个可以支撑大规模竞赛的综合服务平台。

还要感谢担任新生赛出题组的同学们。系统开发过程中，他们从竞赛实战、出题人角度提出诸多具有挑战性的功能需求和修改建议，关于多测试点结果展示、测试数据快速验证、比赛实时榜单交互逻辑等内容都有涉及到。正因为得到了深度用户从各个角度给系统提供建议之后，我才在 PrimeVue、TailwindCSS 的基础之上对前端 UI 展开专门的优化工作，并且也完成了后端微服务架构快速迭代的过程。

另外，我也非常感谢自己在 2024 年独立排查 HustOJ 宕机问题的经历。通过这

次经历可以看出，单体架构对于处理突然出现的流量是有局限性的，因此采用了使用 Spring Cloud Alibaba 以及 RabbitMQ 实现异步解耦的方式来重构。

最后感谢在 2025 年济南大学新生赛中参加系统测试的 270 名同学。你们每次提交的代码、每次提出的建议，都是我技术进步的推动力。这段独立解决大竞赛问题的经历将会是我大学生涯中最令我感到自豪的时刻。